

Hypercomplex neural networks: Exploring quaternion, octonion, and beyond in deep learning

Raghavendra M Devadas^a, Vani Hiremani^b, Preethi^{a,*}, Sowmya T^a, Sapna R^a, Praveen Gujjar^c

^a Manipal Institute of Technology Bengaluru, Manipal Academy of Higher Education, Manipal, India

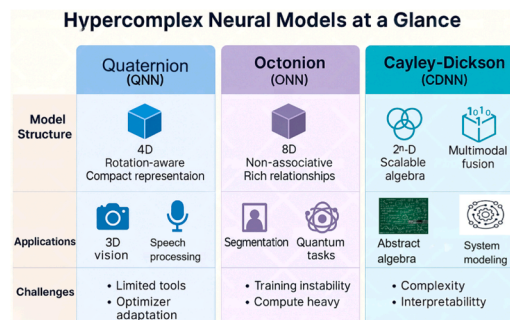
^b Symbiosis Institute of Technology, Symbiosis International (Deemed) University, Pune, India

^c Faculty of Management Studies JAIN (Deemed-to-be University) Bengaluru, India

HIGHLIGHTS

- This review systematically investigates the incorporation of quantum technologies with conventional algorithms such as SVM, KNN, Naïve Bayes, K-Means, and Neural Networks, presenting mathematical frameworks, main results, and application-specific enhancements.
- The research estimates the improvements in accuracy offered by Quantum Support Vector Machines and Quantum KNN over domains, with a critical review of current issues like scalability, noise susceptibility, and limitations of hardware.
- The review outlines future directions as noise-resistant quantum algorithms, effective resource utilization, and increased real-world applications, creating a solid platform for pushing forward with the development of quantum machine learning technologies beyond experimental levels.

GRAPHICAL ABSTRACT



ARTICLE INFO

Keywords:

Hypercomplex neural networks
Quaternion and octonion algebras

ABSTRACT

Hypercomplex Neural Networks (HNNs) represent the next frontier in deep learning, building on the mathematical theory of quaternions, octonions, and higher-dimensional algebras to generalize conventional neural architectures. This review synthesizes cutting-edge methods with their theoretical bases, architectural advancements, and primary applications, tracing the development

* Corresponding author.

E-mail address: preethi.srivathsa@manipal.edu (Preethi).

Cayley-Dickson construction
Higher-dimensional data representation

of hypercomplex mathematics and its implementation in computational models. We distil key advances in quaternion and octonion networks, highlighting their ability to provide compact representations and computational efficiency. Particular attention is given to the unique challenge of non-associativity in octonions—where the order in which numbers are multiplied affects the result—requiring careful design of network operations. The article also discusses training complexity, interpretability, and the lack of standardized frameworks, alongside comparative performance with real- and complex-valued networks. Future directions include scalable algorithm construction, lightweight architectures through tensor decompositions, and integration with quantum-inspired systems using higher-order algebras. By presenting a systematic synthesis of current literature and linking these advances to practical applications, this review aims to equip researchers and practitioners with a clear understanding of the strengths, limitations, and potential of HNNs for advancing multidimensional data modelling.

Specifications table

Subject area	Computer Science
More specific subject area	Neural Networks
Name of the reviewed methodology	Hypercomplex Neural Networks
Keywords	Hypercomplex Neural Networks, Quaternion and Octonion Algebras, Cayley-Dickson Construction, Higher-Dimensional Data Representation
Resource availability	NA
Review question	- What are the mathematical contributions of hypercomplex numbers which make them applicable to machine learning? - What are the application areas, mathematical contributions, and limitations of past works that incorporates Quaternion numbers with Neural networks? - What are the application areas, mathematical contributions, and limitations of past works that incorporates Octonion numbers with Neural networks? - What are the application areas, mathematical contributions, and limitations of past works on Hypercomplex numbers?

Background

Deep learning has revolutionized several domains, including computer vision, natural language processing, and bioinformatics, with efficient extraction and representation of patterns from complex data. Traditional neural networks, which primarily operate in real-valued spaces, often face challenges due to the inherent complexities of multi-dimensional and multi-modal data, such as capturing interdependencies across channels or handling spatiotemporal relationships efficiently. These challenges have motivated researchers to find alternative mathematical frameworks, like hypercomplex numbers, to extend the capabilities of traditional deep learning models [1,2]. This is more applicable to a multidimensional hypercomplex system that supports natural compact coding and processing, such as quaternions, octonions, or other algebras with higher-dimensional bases. In fact, in 1843, Sir W. R. Hamilton created a four-dimensional (Quaternions) generalization of the complex number; it can well model rotation of 3D data sets and some applications involve multi-channel signal processing with two sources and three detectors. The extension of this concept is then to eight dimensions (Octonions); they have more representational capacity but with mathematical complexities of non-associativity, and these properties make hypercomplex numbers especially suitable for tasks such as efficient parameterization and compact representations and modeling interdependencies among data components [3,4]. Fig. 1 represents the hierarchy of hypercomplex numbers. Hypercomplex numbers extend the concept of complex numbers beyond two dimensions. Fig. 2 displays comparison between

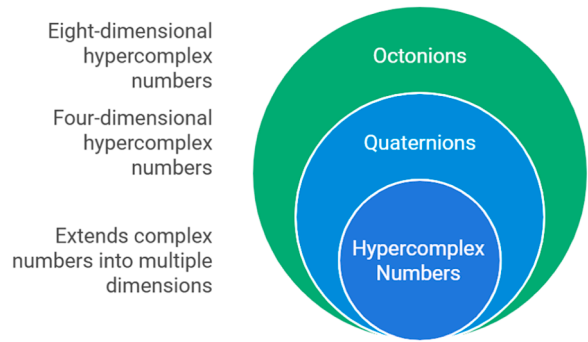


Fig. 1. Hierarchy of hypercomplex numbers.

complex and hypercomplex numbers.

While complex numbers are represented as $(a + bi)$ (where (a) and (b) are real numbers and (i) is the imaginary unit), hypercomplex systems introduce additional dimensions and algebraic structures. Quaternions, for instance, are expressed as $(a + bi + cj + dk)$, where (i, j, k) are imaginary units that follow specific multiplication rules. Octonions extend quaternions into eight dimensions. They are defined as $(a + bi + cj + dk + e\ell + fm + gn + ho)$, where (a, b, c, d, e, f, g, h) are real numbers and (i, j, k, ℓ, m, n, o) are the imaginary units. Beyond quaternions and octonions, there exist other hypercomplex systems such as Sedenions and beyond, which further expand the dimensionality and complexity of algebraic structures. These systems can be used to model more intricate phenomena in physics and computer science [5].

HNNs indeed have emerged as one of the most promising paradigms in deep learning, incorporating hypercomplex algebra into neural architecture. Incorporating hypercomplex neural networks into deep learning architectures leverages their inherent capacity for parameter reduction, thereby facilitating faster training. Moreover, the replacement of conventional real-valued operations with hypercomplex equivalents has the potential to enhance performance on specific tasks by more effectively capturing multidimensional relationships within the data. For example, quaternion-valued networks have indeed outperformed other methods on tasks such as 3D object recognition, speech processing, and biomedical signal analysis owing to their unique ability in processing high-dimensional data with mathematical coherence [6,7]. Despite these advantages, several problems still exist optimization algorithm adaptation to the hypercomplex space is troublesome, models have limited interpretability, and there are no standardized software frameworks [8]. Works on the application of the hypercomplex numbers, such as Quaternions, Octonions, and higher-dimensional algebras like Sedenions, form only the very foundation within machine learning. While some works exist using Quaternion-valued neural networks and an octonion-based model was proposed for 3D vision and signal processing, more general usefulness is yet unexploited systematically. Most related work is bound to narrow tasks like image recognition or geometric learning; relatively few works address general-purpose architecture, optimization techniques, and multi-disciplinary applications. Besides, a side-by-side comparison of various hypercomplex systems in practical machine learning tasks remains absent, leaving big gaps in understanding their full potential and trade-offs. Quantum- and hypercomplex-based machine learning techniques are emerging as powerful tools capable of solving problems that are computationally intensive for traditional methods. These techniques have demonstrated promising results across diverse domains, including healthcare diagnostics (e.g., cancer detection), environmental monitoring (e.g., air quality prediction), natural language processing (e.g., text classification), and computer vision (e.g., medical image analysis). The methodology presented in this work integrates advanced mathematical frameworks—such as quantum-enhanced classifiers, hypercomplex number systems, and novel kernel designs—with machine learning algorithms to improve classification accuracy, computational efficiency, and robustness against noise. While the mathematical foundations are intricate, the central idea is straightforward: we leverage the unique algebraic and computational properties of quantum and hypercomplex systems to process and analyze high-dimensional, complex datasets more effectively than classical methods. This paper reviews existing research that compares performance across domains and identifies the gaps that must be addressed to bridge theoretical advancements with real-world implementation. While HNNs offer theoretical richness, their adoption is hindered by high training complexity, absence of standardized implementation frameworks, and limited empirical benchmarks. Addressing these challenges will require interdisciplinary efforts to simplify architectures, improve reproducibility, and enhance accessibility for both AI practitioners and mathematical computing researchers.

Real-world use cases of hypercomplex neural networks

- **Biomedical Diagnostics – Color Image Feature Extraction:** In image processing applications like color palmprint recognition, the Octonion Neural Networks (ONNs) are utilized to maintain inter-channel relationships through pixel values being encoded into eight-dimensional octonion values. This improves feature extraction compared to real-valued networks and renders ONNs appropriate for biomedical imaging applications [57].
- **Synchronization in Octonion Recurrent Neural Networks with Time-Varying Delays:** Octonion-valued recurrent neural networks (OVNNs) were efficiently utilized for modeling and synchronizing time-varying dynamical systems with time delay—control systems and network behavior—showing fixed-time synchronization under real-world conditions of uncertainty [58].

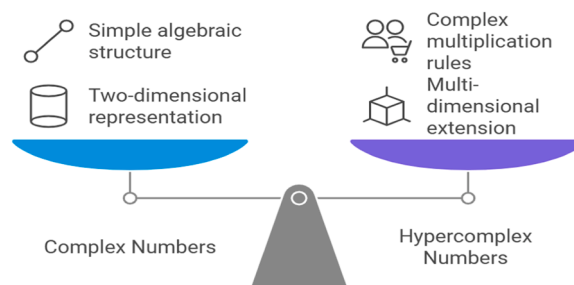


Fig. 2. Comparing dimensional and structural complexity.

- **Quaternion RNNs in Speech Recognition Tasks:** Quaternion recurrent models such as Quaternion RNNs (QRNN) and Quaternion LSTMs (QLSTM) have demonstrated enhanced performance and fewer parameters in automatic speech recognition with performance as good as or even better than real-valued models with up to $3.3 \times$ fewer parameters.

This study aims to address the research gaps by providing a comprehensive review and synthesis of the use of hypercomplex numbers in machine learning. The importance of this work can be summarized as follows:

- **Advancing the State of Knowledge:** This work establishes a foundation for researchers to better understand the mathematical, computational, and practical aspects of hypercomplex algebras in machine learning by consolidating and analyzing the existing literature.
- **Underexplored Areas:** The research identifies key gaps, such as the lack of studies on higher-order algebras (e.g., Sedenions) and the need for efficient, scalable algorithms, thereby providing direction for future research.
- **Fostering Interdisciplinary Innovation:** Hypercomplex algebras inherently model multi-dimensional relationships, making them suitable for applications in the fields of physics-informed AI, bioinformatics, and quantum machine learning. This paper points out their potential in bridging disciplines and generating innovation.
- **The paper begins by giving a comparative overview of Hypercomplex system, further this work considers the mathematical foundations behind the hypercomplex numbers. From there, the development of quaternion and octonion-based neural networks will be discussed and reviewed, underlining key architectural innovations as well as important applications in several fields. Finally, the review highlights open research challenges, like the necessity for scalable training methods and further investigation of higher order algebras and discusses possible synergies with emerging fields like quantum computing.**

Through this review, by synthesizing past and recent developments, a clearer understanding of hypercomplex neural networks, their capabilities, and limitations will be provided to both researchers and practitioners. The insights presented herein shall therefore guide future research and the associated innovation in this emerging machine learning field.

Method details

The methodology adopted in this work follows a review-based analytical approach, supported by performance comparisons from existing literature. Our study focuses on four main algorithmic families:

1. **Quantum Support Vector Machines (QSVMs)** – used for classification tasks by leveraging quantum kernels to map input data into higher-dimensional Hilbert spaces for improved separability.
2. **Quantum k-Nearest Neighbors (QKNN)** – employs quantum distance metrics to classify data points based on their proximity in quantum state space, reducing computational complexity in high-dimensional problems.
3. **Quantum Neural Networks (QNNs)** – combines classical neural network architectures with quantum circuits, enabling hybrid models to exploit quantum parallelism for faster learning.
4. **Hypercomplex Number Systems (HNS)-based Models** – uses multi-dimensional algebraic systems (e.g., quaternions, octonions) for compact data representation and parameter reduction in deep learning models.

The review process involved systematically collecting research papers from IEEE Xplore, Springer, and other databases, categorizing them by application domain, mathematical contribution, key findings, and limitations. Figures and comparative tables were then generated to visualize the distribution of applications, highlight performance variations, and identify recurring challenges such as noise sensitivity, scalability constraints, and hardware dependency.

Below are the review questions framed for this study

Table 1
Overview of hypercomplex systems.

Hypercomplex number system	Dimensionality	Key properties	Mathematical operations	Applications in ML	Challenges
Quaternions	4D	Non-commutative multiplication; Compact representation of 3D rotations [1,9]	Addition, multiplication, conjugation, norm	RGB-D image recognition [3]; Speech processing [10]; Electromyography signal analysis [11]	Requires specialized optimization; Limited tool/library support
Octonions	8D	Non-associative multiplication; Generalization of quaternions [8]	Addition, multiplication, conjugation, norm	3D rotations [12]; Modeling higher-order relationships in data [13]	Increased computational overhead; Mathematical complexity
Clifford Algebras [14]	Variable	Associative; Models geometry and spacetime [15]	Addition, multiplication, wedge product	Robotics [16]; Physics-informed machine learning	Complex implementation; Computational expense in higher dimensions
Cayley-Dickson Extensions	2 ⁿ D	Extends hypercomplex systems recursively [17]	Addition, multiplication, norm	Potential for extremely high-dimensional data tasks [18]	Escalating complexity; Difficult to interpret results

- What are the mathematical contributions of hypercomplex numbers which make them applicable to machine learning?
- What are the application areas, mathematical contributions, and limitations of past works that incorporates Quaternion numbers with Neural networks?
- What are the application areas, mathematical contributions, and limitations of past works that incorporates Octonion numbers with Neural networks?
- What are the application areas, mathematical contributions, and limitations of past works on Hypercomplex numbers?

Table 1 provides a comparative overview of key hypercomplex number systems, showing their mathematical properties, operations, applications in machine learning, and associated challenges. It is a concise reference for understanding their relevance and trade-offs in neural network design.

Question 1. What are the mathematical contributions of hypercomplex numbers which make them applicable to machine learning?

To address this question, we discuss the mathematical foundation of hypercomplex numbers: their key properties and operations, as well as algebraic structures that make them applicable to machine learning. Hypercomplex numbers are an extension of the complex number system into higher dimensions, such as quaternions and octonions, which can describe multi-dimensional data in a more compact and efficient way. Such algebras offer a potent model of relating data features and carry advantages over real-valued networks, such as a reduced parameter set and the ease to model multi-dimensional data in an unforced manner.

Quaternions

Quaternions (H) are an extension of complex numbers, introduced by Hamilton in 1843, that consist of four components. A general quaternion $q \in H$ can be written as:

$$q = \alpha + \beta i + \gamma j + \delta k \quad (1)$$

Where:

- $\alpha, \beta, \gamma, \delta \in \mathbb{R}$ are real numbers,
- i, j, k are the quaternion units, satisfying the following multiplication rules:

$$i^2 = j^2 = k^2 = ijk = -1, \quad ij = k, \quad jk = i, \quad ki = j,$$

with reversed products carrying a negative sign (e.g., $ji = -k$).

The multiplication of two quaternions $q_1 = \alpha_1 + \beta_1 i + \gamma_1 j + \delta_1 k$ and $q_2 = \alpha_2 + \beta_2 i + \gamma_2 j + \delta_2 k$ is defined as:

$$q_1 q_2 = (\alpha_1 \alpha_2 - \beta_1 \beta_2 - \gamma_1 \gamma_2 - \delta_1 \delta_2) + (\alpha_1 \beta_2 + \beta_1 \alpha_2 + \gamma_1 \delta_2 - \delta_1 \gamma_2) i + (\alpha_1 \gamma_2 - \beta_1 \delta_2 + \gamma_1 \alpha_2 + \delta_1 \beta_2) j + (\alpha_1 \delta_2 + \beta_1 \gamma_2 - \gamma_1 \beta_2 + \delta_1 \alpha_2) k \quad (2)$$

The quaternion conjugate is defined as:

$$q^* = \alpha - \beta i - \gamma j - \delta k \quad (3)$$

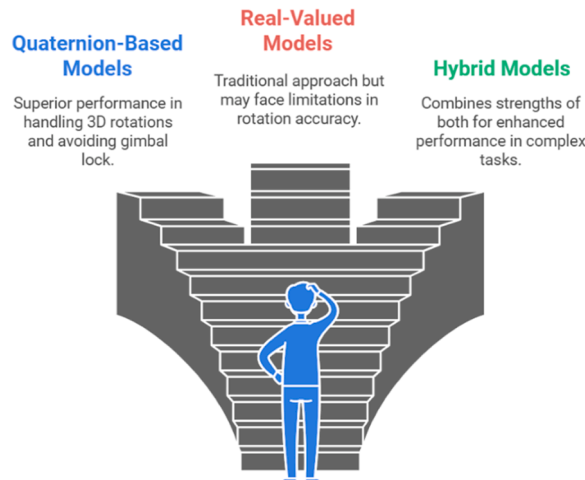


Fig. 3. A comparison between 3 neural network models.

The norm (magnitude) of a quaternion is given by:

$$|q| = \sqrt{\alpha^2 + \beta^2 + \gamma^2 + \delta^2} \quad (4)$$

Unlike all other non-commutative structures, the multiplication of quaternions does not fulfill the commutative property; hence, it is important what the order of multiplication is. It particularly serves well in capturing symmetries under rotations in 3D space, which heavily depends on the order of transformation. Quaternion neural networks (QNNs) are a variant of traditional neural nets, using quaternions instead of real numbers for weights and activations, thus intrinsically capturing information about rotation, which can be used effectively for 3D spatial transform tasks [19]. Fig. 3 displays a comparison between three types of models.

Fig. 3 compares the performance and characteristics of three deep learning models—a standard real-valued network, a complex-valued network, and a quaternion-valued network—on an identical task. It highlights the quaternion model's advantages, such as more compact parameterization and the ability to naturally encode 3D rotational information, which together yield improved efficiency or accuracy over the real and complex counterparts.

The paper [20] proposed revising neural networks for 3D point cloud processing into rotation-equivariant quaternion neural networks (REQNNs), which improved rotation robustness and maintained feature invariance to input permutations as shown in Fig. 4.

As illustrated in Fig. 4, the rotation-equivariant quaternion neural network architecture explicitly encodes rotational symmetry for 3D data. In this design, if a point cloud input is rotated in space, the network's internal feature representations rotate accordingly, which makes the model's output immune to arbitrary orientations of the input. This property, along with built-in invariance to the ordering of points, is crucial for robust 3D object recognition

Octonions

Octonions (O) are an eight-dimensional extension of quaternions and are part of the Cayley-Dickson construction. An octonion $o \in O$ is represented as:

$$o = \mu + \nu_1\lambda_1 + \nu_2\lambda_2 + \nu_3\lambda_3 + \nu_4\lambda_4 + \nu_5\lambda_5 + \nu_6\lambda_6 + \nu_7\lambda_7 \quad (5)$$

Where:

- $\mu, \nu_1, \dots, \nu_7 \in \mathbb{R}$ are real coefficients,
- $\lambda_0, \lambda_1, \dots, \lambda_7$ are the basis elements of the octonion algebra, satisfying specific multiplication rules.

The multiplication rules for the octonion basis elements are non-associative but satisfy the following.

$$\lambda_i^2 = -1 \text{ for } i \geq 1,$$

$$\lambda_i\lambda_j = -\lambda_j\lambda_i \text{ for } i \neq j.$$

The norm of an octonion is:

$$|o| = \sqrt{\mu^2 + \nu_1^2 + \nu_2^2 + \nu_3^2 + \nu_4^2 + \nu_5^2 + \nu_6^2 + \nu_7^2} \quad (6)$$

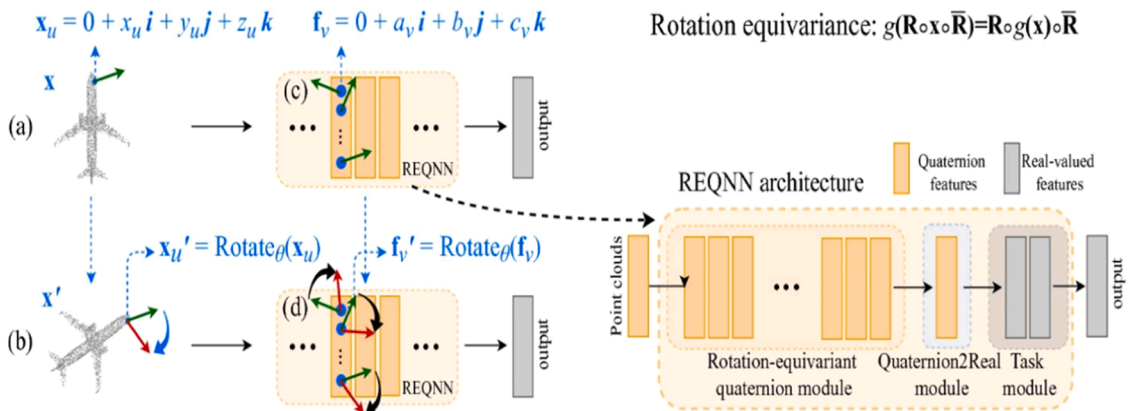


Fig. 4. Rotation-equivariant quaternion neural network architecture [20].

Expanded explanation of octonion multiplication

Unlike quaternions, octonions O form a non-associative but alternative division algebra generated by one real unit and seven imaginary units $\{e_1, \dots, e_7\}$. The rules for multiplication are best illustrated using the Cayley multiplication table (see Fig. 5) [9] or its equivalent Fano plane diagram. Each row and column correspond to one of the basis units, and the entries define the product $e_i e_j$.

Since octonion multiplication is non-associative (i.e., $(ab)c \neq a(bc)$ in general), order of operation must be preserved explicitly in computation graphs. That affects ONN structure: during forward propagation, Hamilton-like products are computed in a determined order, and during backpropagation, gradients must preserve order of operations carefully so they do not update parameters wrongly. Algebraic constraints incur computational costs but add expressiveness to ONNs.

Octonions are particularly useful for applications where hierarchical relationships or non-associative algebra are necessary, such as higher-dimensional embeddings in neural networks or specific tasks in physics. An Octonion Neural Network, or ONN, can be built by simply replacing the standard real-valued weights and activations with the respective octonion-valued versions. Some of the basic building blocks of an ONN include:

- **Input Layer:** All inputs may be represented as an octonion and thus feature encodings as multidimensional as desired.
- **Hidden Layers:** Neurons in hidden layers carry out computations over octonions and make use of octonion multiplication along with activations appropriate for octonions.
- **Output Layer:** The output can likewise be represented in the octonion domain, thereby enriching the expressiveness of the predicted values through higher-dimensional feature encoding.

The authors of [21] discuss the construction and implementation of Deep Octonion Networks (DONs) for image classification tasks, comparing their performance with Real NNs, DCNs, and DQNs, and explaining their success through multi-task learning as shown in Fig. 6. Also Fig. 7 indicates the performance of DONs as compared to other models.

Fig. 6 outlines the network's octonion-valued layers and illustrates how multiple learning tasks or outputs can be integrated (a multi-task learning component) within the architecture. Following Wu et al.'s Deep Octonion Networks, the components are structured as OctonionConv $\rightarrow$ Octonion BatchNorm (OctonionBN) $\rightarrow$ ReLU, and stacked into residual blocks (two convs in a block) prior to global pooling and a dense classifier. DONs are trained with softmax cross-entropy and stochastic gradient descent with Nesterov momentum = 0.9; Authors used use batch size = 64 and staged learning-rate schedule for 120 epochs (warm-up at 0.01, $\alpha \times 10$ increase at middle of training, then drop to stabilize). By leveraging octonion algebra for weight and activation representations, the DON can compactly fuse multi-channel image information, and the inclusion of a multi-task structure helps explain its superior performance on image classification tasks.

Fig. 7 compares the image classification performance of the Deep Octonion Network (DON) against three other network types: a traditional real-valued CNN, a deep complex-valued network (DCN), and a deep quaternion network (DQN). The plot shows that the DON achieves higher accuracy than the real, complex, and quaternion models on the same benchmark, underscoring the effectiveness of octonion-based representations. Notably, the octonion network reaches comparable or better accuracy with a more compact model, highlighting improvements in both learning efficiency and predictive performance.

Clifford algebra

Clifford algebras generalize both complex numbers and quaternions to higher dimensions and can be applied in machine learning for tasks involving geometric and spatial relationships [15]. A Clifford algebra $Cl_{p,q}$ can be written as:

$$Cl_{p,q} = \mathbb{R}^n \oplus (\mathbb{R}^n \otimes \mathbb{R}^m) \quad (7)$$

Where,

- p and q are the number of vector spaces with positive and negative signatures, respectively.

	1	e_1	e_2	e_3	e_4	e_5	e_6	e_7
1	1	e_1	e_2	e_3	e_4	e_5	e_6	e_7
e_1	e_1	-1	e_3	$-e_2$	e_5	$-e_4$	$-e_7$	e_6
e_2	e_2	$-e_3$	-1	e_1	e_6	e_7	$-e_4$	$-e_5$
e_3	e_3	e_2	$-e_1$	-1	e_7	$-e_6$	e_5	$-e_4$
e_4	e_4	$-e_5$	$-e_6$	$-e_7$	-1	e_1	e_2	e_3
e_5	e_5	e_4	$-e_7$	e_6	$-e_1$	-1	$-e_3$	e_2
e_6	e_6	e_7	e_4	$-e_5$	$-e_2$	e_3	-1	$-e_1$
e_7	e_7	$-e_6$	e_5	e_4	$-e_3$	$-e_2$	e_1	-1

Fig 5. Fano plane representation of octonion multiplication rules. Each oriented line encodes a cyclic multiplication relation among three imaginary units with orientation determining the sign.

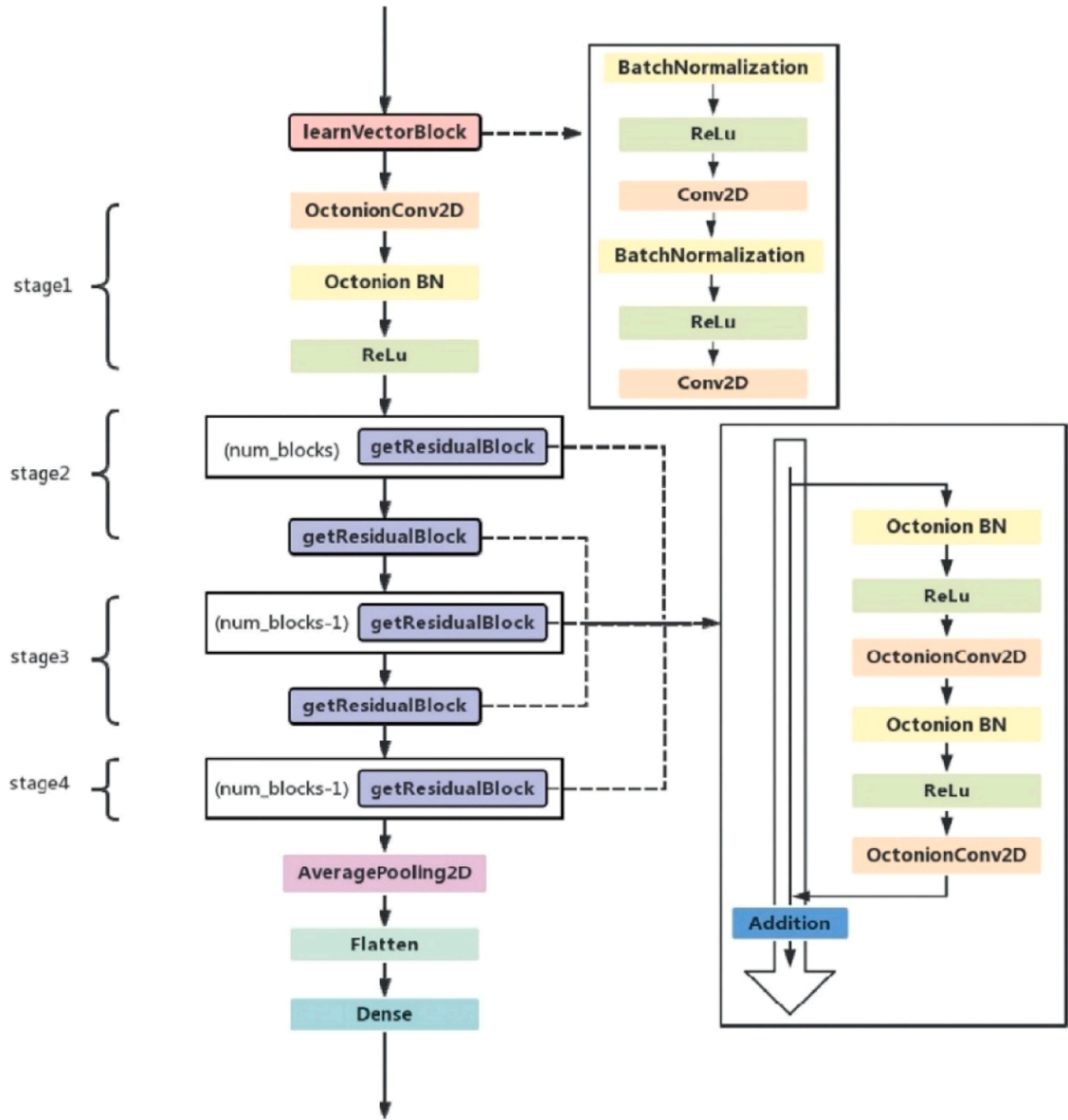


Fig. 6. Deep Octonion Network architecture with multi-task design [21].

- These algebras are widely used for applications in physics, robotics, and geometry.

Clifford algebra multiplication follows specific rules related to the geometry of the underlying vector spaces. The geometric product xy of two vectors x and y in a Clifford algebra is defined as:

$$xy = x \cdot y + x \wedge y \quad (8)$$

Where $x \cdot y$ is the inner product (scalar) and $x \wedge y$ is the outer product (bivector). This algebraic structure allows for modeling rotations, reflections, and other geometric transformations, making it useful in applications such as computer vision and robotic motion planning. For a neural network, the scalar component $x \cdot y$ can be interpreted to be quantifying magnitude-based similarity of the features, like normal dot-product calculation in fully connected layers. The bivector component $x \wedge y$ is represented using oriented area information, something that can be attained as ordered cross-feature interaction among convolutional or attention layers. The composed representation enables layers to have scalar relations along with geometric transforms such as rotations and reflections and is hence useful in tasks involving computer vision, robotics, and 3D scene understanding. In neural networks utilizing Clifford algebras, a key innovation is the incorporation of learnable metrics [22]. These metrics dynamically adapt the internal geometry of the network, enhancing its ability to process data in spaces with varying geometric properties.

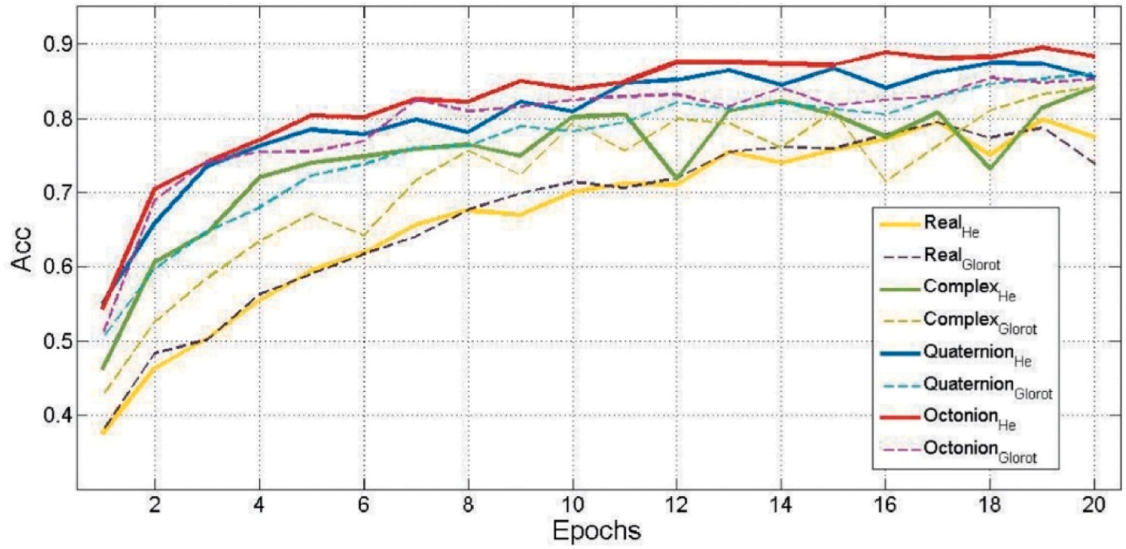


Fig. 7. Performance of the DON versus real, complex, and quaternion models [21].

A learnable metric M is initialized as a perturbed symmetric matrix:

$$M = Q + \epsilon \cdot (R + R^T) \quad (9)$$

Where,

- Q : Diagonal metric (e.g., Euclidean),
- R : Random matrix,
- ϵ : Perturbation coefficient.

The metric M can be decomposed into eigenvalues and eigenvectors for efficient computation:

$$M = P^{-1} \Delta P \quad (10)$$

Where:

P : Matrix of eigenvectors, Δ : Diagonal matrix of eigenvalues.

Clifford group equivariant neural networks leverage the geometric product to capture complex interactions between data components. For multivectors y_1 and y_2 , the geometric product in a neural network layer is defined as:

$$G_\psi(y_1, y_2)^{(m)} = \sum_{a=0}^n \sum_{b=0}^n \psi_{abm} \cdot (y_1^{(a)} \cdot y_2^{(b)})^{(m)} \quad (11)$$

Where,

- y_1, y_2 : Multivectors,
- ψ_{abm} : Learnable parameters,
- m : Target grade.

The parameters ψ_{abm} act as the learnable weights that determine the interaction between components along basis e_a interact with components along basis e_b within the m^{th} output channel or feature map. These parameters are optimized during training using backpropagation, enabling the network to learn the geometric interactions to the current task. Architecturally, ψ_{abm} can be represented as a structured weight tensor, in which indices a and b is indexed to basis interactions and m indexes the output channels. This representation allows both levels of the network to learn the magnitude and geometric orientation of features, which produce richer representations than non-real-valued layers.

The normalization operation ensures numerical stability and is defined as:

$$\text{Norm}(z^{(t)}) = \frac{z^{(t)}}{\sigma(\beta_t \cdot (R(z^{(t)}) - 1) + 1)} \quad (12)$$

In [23], authors introduced Clifford Group Equivariant Graph Neural Networks (CG-EGNNs), which explores the powerful expressive ability of Clifford algebras on geometric graph learning. The framework has a high-order mechanism for message passing in which one can encode scalar and vector information, but it is also possible for encoding planes and volumes as illustrated in Fig. 7.

Fig. 8 provides an overview of the CG-EGNN architecture introduced by Ali et al. [23], which incorporates Clifford algebra into graph neural networks. It shows how the network's message-passing layers encode various grades of geometric features: beyond just scalars and vectors, the CG-EGNN can represent planes and volumes (bivectors, trivectors, etc.) as part of its node feature transformations. By using learnable metric matrices and the Clifford geometric product, this design ensures equivariance under Euclidean transformations (rotations/reflections) in the input graph data. The illustrated architecture underlines the enhanced expressive power gained by embedding high-order geometric information in the network.

Cayley-Dickson construction

The Cayley-Dickson construction is an iteration algorithm to obtain hypercomplex numbers with starting from the real number system. Starting from the real numbers $\mathbb{R}$, one gets first the complex numbers $\mathbb{C}$, then quaternions $\mathbb{H}$, then octonions $\mathbb{O}$, and so on. Doubling dimensionality at each step, the resulting algebra produces more complicated multiplication rules. For example, the Cayley-Dickson extension of the quaternions produces the octonions. The general form for the Cayley-Dickson construction of the next algebra A_{n+1} from A_n is given by:

$$A_{n+1} = A_n \oplus A_n \quad (13)$$

Cayley-Dickson Neural Networks (CDNN), on the other hand, incorporate hypercomplex number systems that improve the capabilities of more traditional neural network architectures. These networks use Cayley-Dickson algebras, which extend complex number systems to higher dimensions. This allows for richer representations and operations within neural computations. Fig. 8 displays a few characteristics of CDNN.

The schematic (Fig. 9) highlights the fundamental features of neural networks built on the Cayley-Dickson construction. It summarizes how each iterative application of the Cayley-Dickson doubling process extends the dimensionality of number systems (real $\rightarrow$ complex $\rightarrow$ quaternion $\rightarrow$ octonion $\rightarrow \dots$) and how those higher-dimensional algebras can be integrated into network layers. Important characteristics illustrated include the preservation of algebraic properties (like a generalized notion of norm and conjugation) in each extension and the ability of CDNNs to naturally encode higher-dimensional data without exploding parameter count.

In one of the works authors introduce a framework for extreme learning machines (ELMs) based on Cayley-Dickson algebras, demonstrating their effectiveness in color image auto-encoding tasks using the CIFAR dataset, where they outperform traditional real-valued ELMs [24]. The study highlights the advantages of hyper-complex algebras in enhancing machine learning models' performance on complex data structures.

The Cayley-Dickson norm defines the magnitude of an element in a Cayley-Dickson algebra as the sum of the squared magnitudes of its components, ensuring that the product of an element and its conjugate equals the square of its norm, which is critical for preserving division-like properties.

$$m(v) = vv^* = (x, y)(x^*, -y) = m(x) + \delta_k m(y) \quad (14)$$

The authors introduce a real-valued representation for matrix multiplication within Cayley-Dickson algebras. This is done by defining left multiplication matrices (Ψ_L) and extend hypercomplex-valued computations to fit into real-valued frameworks, which can be easily implemented in neural network architectures.

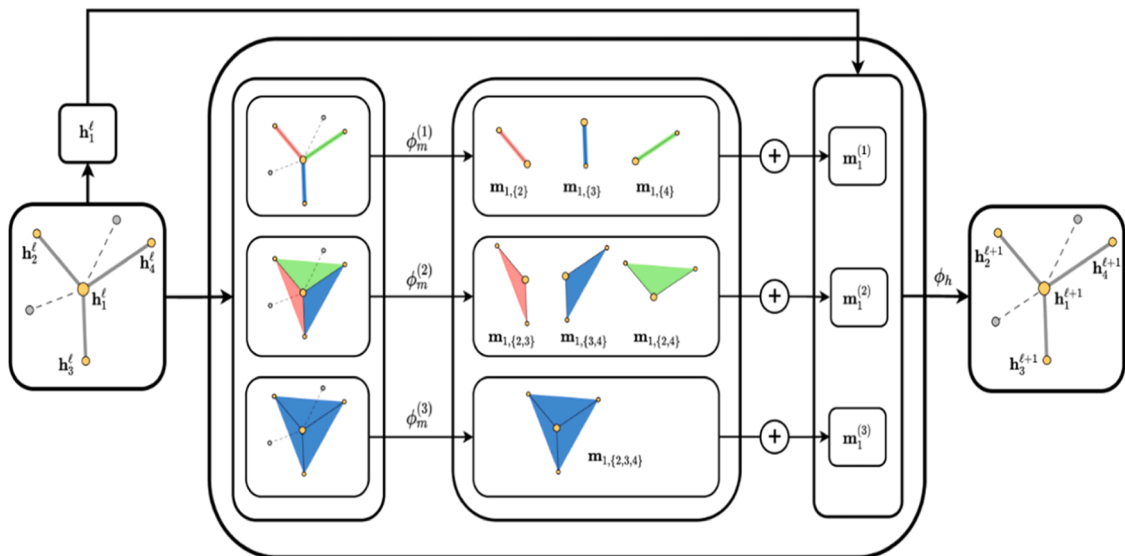


Fig. 8. Clifford group equivariant graph neural network illustration [23].

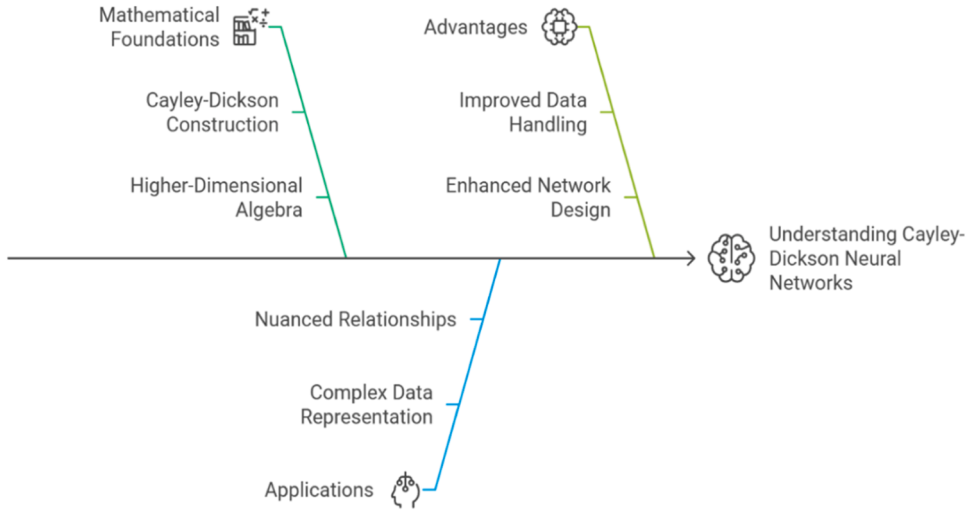


Fig. 9. Key characteristics of a Cayley–Dickson neural network.

$$R_{ij} = \sum_{m=1}^q P_{im} \cdot Q_{mj} \quad (15)$$

rewritten as real-valued algebra terms:

$$\psi(R) = \Psi_L(P) \cdot \psi(Q) \quad (16)$$

where,

$\psi(R)$: Real-valued representation of R ,

$\Psi_L(P)$: Left multiplication matrix in real-valued space.

The Cayley–Dickson construction extends a high dimensionality of algebra currently known while preserving some significant

Algorithm 1

Master workflow for hypercomplex neural networks.

Inputs: Training set $\{(x_n, y_n)\}_{n=1..N}$; algebra $A \in \{R, C, H, O, A_n\}$;
network F_θ defined over A ; optimizer Opt ; regularizer $\Omega(\cdot)$; epochs E ; batch size B

Output: Trained parameters θ^*

```

1      Initialize parameters  $\theta$  (weights and biases) in algebra  $A$ 
2      for epoch = 1..E do
3          for each minibatch  $B \subset \{1..N\}$  do
4              # — Embedding / Input preparation —
5               $X \leftarrow \text{EMBED\_TO\_ALGEBRA}(\text{BATCH}(x_n), A)$ 
6
7              # — Forward pass —
8               $h \leftarrow X$ 
9              for each layer  $L$  in  $F_\theta$  do
10                  $h \leftarrow \text{MULTIPLY\_A}(W_L, h) + b_L$  # algebra-specific linear op
11                  $h \leftarrow \text{ACTIVATE\_SPLIT}(h)$  # apply nonlinearity per component
12                  $h \leftarrow \text{NORMALIZE\_IF\_NEEDED}(h)$  # optional: component-wise BN
13             end for
14              $\hat{Y} \leftarrow \text{HEAD}(h)$  # final prediction layer
15
16             # — Loss —
17              $L \leftarrow (1/|B|) * \sum_{n \in B} \mathcal{L}(\hat{Y}_n, y_n) + \lambda \Omega(\theta)$ 
18
19             # — Backward pass —
20             # Use real-valued embedding of  $A$  so autodiff computes grads reliably
21             # Keep a fixed multiplication order for non-associative algebras  $(O, A_n)$ 
22              $G \leftarrow \text{AUTODIFF}(L, \theta, \text{using REAL\_EMBEDDING}(A))$ 
23
24             # — Parameter update —
25              $\theta \leftarrow \text{Opt}(\theta, G)$ 
26         end for
27     end for
28     return  $\theta^*$ 

```

properties, making it more powerfully applicable to any higher dimensional data in learning, machine. One particular very interesting application of that is shown in the problem statement is solving the Least Squares Problem in Cayley-Dickson algebra, which simplifies the optimization process in complex hyperspaces. This Least Squares Problem for Cayley-Dickson formulates the difference of observed data and model's predictions in the hypercomplex space of the algebra to be minimized. The least squares problem in the Cayley-Dickson algebra for a hypercomplex-valued matrix P is expressed as:

$$\min |BP - S|_F, \quad P \in \mathbb{B}_k^q \times r \quad (17)$$

The Cayley-Dickson least squares problem is reformulated into real-valued linear algebra to facilitate computation. Using the real-valued representations ψ and left multiplication matrix Ψ_L , the solution is given by:

$$P = \psi^{-1}(\Psi_L(B)^\dagger \Psi(S)) \quad (18)$$

To make reproducibility stronger, we first introduce a unified master workflow (see [Algorithm 1](#)) to train hypercomplex neural networks. This summary illustrates how inputs are embedded into the selected algebra, how algebra-dependent multiplications and activations are executed in the forward pass, and how gradients are calculated in the backward pass with real-valued embeddings. The following algorithms ([Algorithms 2–5](#)) present the main subroutines—like the Cayley-Dickson multiplication and its gradient—utilized within this process.

[Algorithm 1](#) illustrates the end-to-end training pipeline for hypercomplex neural networks over a variety of algebras (R,C,H,O,An). The process starts from the initialization of the weights and biases in the selected algebra, then embedding the input data into the corresponding hypercomplex representation. In the forward pass, every layer performs an algebra-specific multiplication rule—e.g., the Hamilton product in quaternions or Cayley-Dickson product in higher-order extensions—combined with split nonlinear activations separately on every component. The backward pass uses real-valued embeddings of the hypercomplex operations, and thus can use standard automatic differentiation tooling while preserving the well-defined order of multiplication in non-associative algebras such as octonions. This framework guarantees both forward calculations and gradient propagation are unambiguously defined across algebras. Lastly, parameters are optimized using the selected optimizer, finishing the training loop. By giving this high-level schematic, [Algorithm 1](#) unifies the training process into a reproducible workflow. Subsequent algorithms ([Algorithms 2–5](#)) then specify the subroutines called herein, such as Cayley-Dickson multiplication and gradient calculation, thus closing the gap between abstract workflow and its concrete instantiations.

[Algorithm 2](#) provides Cayley-Dickson Neural Network Recursive Construction & Training loop.

[Algorithm 2](#) presents a neural architecture based on Cayley-Dickson algebra which recursively maps the real numbers to 2^n -dimensional hypercomplex spaces. Recursive building allows CDNNs algebraic scaling of high-dimensional applications. The forward

Algorithm 2

CDNN training loop.

Input: Training set $D = \{(C_{in}, y)\}$, learning rate η , epochs E , depth n
Output: Trained CDNN weights W_c , biases b_c

```

1: function CD_CONSTRUCT(n):
2:   if  $n == 0$ :
3:     return REAL_NUMBER_SPACE
4:   else:
5:     prev_space  $\leftarrow$  CD_CONSTRUCT( $n - 1$ )
6:     return (prev_space, prev_space) with defined CD_MULTIPLICATION_RULE
7:   end if
8: end function
9: Initialize  $W_c$ ,  $b_c$  in  $2^n$ -D space from CD_CONSTRUCT( $n$ )
10: for epoch = 1 to  $E$  do
11:   for each  $(C_{in}, y)$  in  $D$  do
12:     // Forward pass
13:      $C_{mul} \leftarrow$  CD_MULTIPLY( $C_{in}, W_c$ ) [24]
14:      $C_{act} \leftarrow$  ACT_FN( $C_{mul} + b_c$ )
15:      $y_{hat} \leftarrow$  SOFTMAX( $C_{act}$ )
16:
17:     // Loss computation
18:      $L \leftarrow$  CROSS_ENTROPY( $y_{hat}, y$ )
19:
20:     // Backward pass
21:      $\delta \leftarrow$  GRAD_LOSS( $y_{hat}, y$ )
22:      $\delta_{act} \leftarrow$  GRAD_ACT_FN( $\delta, C_{act}$ )
23:      $\partial W_c \leftarrow$  CD_GRAD( $C_{in}, \delta_{act}$ ) [24]
24:      $\partial b_c \leftarrow$  SUM( $\delta_{act}$ )
25:
26:     // Parameter update
27:      $W_c \leftarrow W_c - \eta * \partial W_c$ 
28:      $b_c \leftarrow b_c - \eta * \partial b_c$ 
29:   end for
30: end for

```

pass uses Cayley–Dickson multiplication, with output normalization and non-linear activation. The backward pass computes gradients in the augmented algebra space to make it possible for learning to happen for very high-dimensional feature representations. CDNNs are optimally suited for multimodal fusion, high-dimensional representation, and algebraically structured learning but suffer from interpretability and numerical stability issues at higher orders.

To illustrate the algorithmic implications of the Cayley–Dickson construction, this section contrasts a standard real-valued Extreme Learning Machine (ELM) with its complex and quaternionic counterparts, where the latter two are obtained through one and two successive Cayley–Dickson doublings, respectively.

Real-valued ELM (Baseline)

Given training data $\{(x_n, y_n)\}_{n=1}^N$ with $X_n \in \mathbb{R}^d$ a single hidden-layer ELM with L hidden units selects random weights $W \in \mathbb{R}^{L \times d}$ and biases $b \in \mathbb{R}^L$. The hidden layer output matrix $H \in \mathbb{R}^{N \times L}$ is computed as

$$H_{n,i} = g(w_i^\top x_n + b_i) \quad (19)$$

Where, $x_n \in \mathbb{R}^d$: input feature vector for the n -th sample.

$w_i \in \mathbb{R}^d$: weight vector of the i -th hidden unit.

$b_i \in \mathbb{R}$: bias term of the i -th hidden unit. $g(\cdot)$: activation function (e.g., sigmoid, ReLU).

$H_{n,i}$: output of the i -th hidden node for the n -th input.

The output weights $\beta \in \mathbb{R}^L$ are obtained by solving the regularized least-squares problem:

$$\beta = \underset{\beta}{\operatorname{argmin}} \|H\beta - Y\|_2^2 + \lambda \|\beta\|_2^2 = (H^\top H + \lambda I)^{-1} H^\top Y \quad (20)$$

Where,

$H \in \mathbb{R}^{N \times L}$: hidden layer output matrix (N samples, L hidden units).

$Y \in \mathbb{R}^N$: target output vector.

$\beta \in \mathbb{R}^L$: output weight vector.

λ : regularization parameter.

I : identity matrix of size $L \times L$.

Complex-valued ELM (C-ELM)

In the first Cayley–Dickson doubling, inputs are represented as complex vectors:

$$x_n = u_n + iv_n \in \mathbb{C}^d,$$

with weights, $w_i = a_i + ib_i \in \mathbb{C}^d$, $b_i = p_i + iq_i \in \mathbb{C}$

The pre-activation for the i -th hidden unit is given by the complex inner product:

$$z_{n,i} = w_i^H x_n + b_i \quad (21)$$

Where,

$x_n = u_n + iv_n \in \mathbb{C}^d$: input vector with real part u_n , imaginary part v_n .

$w_i = a_i + ib_i \in \mathbb{C}^d$: weight vector with real part a_i , imaginary part b_i .

$b_i = p_i + iq_i \in \mathbb{C}$: complex bias.

w_i^H : Hermitian transpose (conjugate transpose).

$z_{n,i} \in \mathbb{C}$ pre-activation for the i -th hidden unit.

When implemented on real-valued hardware, this is computed using the block embedding:

$$w_i^H x_n = (a_i^\top u_n + b_i^\top v_n) + i(a_i^\top v_n - b_i^\top u_n) \quad (22)$$

Where,

$a_i, b_i \in \mathbb{R}^d$: real and imaginary parts of the weight vector $u_n, v_n \in \mathbb{R}^d$: real and imaginary parts of the input vector.

The activation function $g: \mathbb{C} \rightarrow \mathbb{C}$ may be defined in a split manner: $g(z) = g_R(\operatorname{Re} z) + i g_I(\operatorname{Im} z)$, or based on modulus–phase preservation.

Quaternion-valued ELM (H-ELM)

Applying the Cayley–Dickson construction a second time yields the quaternion algebra H . Each input is represented as:

$$x_n = x_n^{(r)} + x_n^{(i)} i + x_n^{(j)} j + x_n^{(k)} k \in H^d, \quad (23)$$

$x_n^{(r)}, x_n^{(i)}, x_n^{(j)}, x_n^{(k)} \in \mathbb{R}^d$: real, i -, j -, and k -components of the quaternion input. i, j, k : quaternion units satisfying Hamilton's rules. with corresponding weights:

$$w_i = a_i + b_i i + c_i j + d_i k \in \mathbb{H}^d,$$

and $b_i \in H$.

The pre-activation is obtained through the Hamilton product $\otimes$, applied component-wise across the d-dimensional feature vector:

$$z_{n,i} = \sum_{m=1}^d (w_{i,m} \otimes x_{n,m}) + b_i \quad (24)$$

Where,

$w_{i,m} \in H$: quaternion weight for the m-th feature of the i th hidden unit.

$x_{n,m} \in H$: quaternion input component.

$b_i \in H$: quaternion bias.

$\otimes$: Hamilton product.

$z_{n,i} \in H$: pre-activation quaternion output.

A quaternion-compatible nonlinearity is then applied, such as a split activation over the four components.

For solving the output weights, the quaternion matrix $H \in \mathbb{H}^{N \times L}$ embedded into a real matrix

$$\tilde{H} \in \mathbb{R}^{N \times 4L}$$

containing the real, i -, j -, and k - components, after which the real-valued ridge regression is solved:

$$\tilde{\beta} = (\tilde{H}^T \tilde{H} + \lambda I)^{-1} \tilde{H}^T \tilde{Y} \quad (25)$$

Where,

$H \in \mathbb{H}^{N \times L}$: quaternion hidden layer output matrix.

$\tilde{H} \in \mathbb{R}^{N \times 4L}$: real embedding of HHH, with four channels per quaternion component.

$\tilde{Y} \in \mathbb{R}^{N \times 4}$: real embedding of the target output.

$\tilde{\beta} \in \mathbb{R}^{4L}$: real embedding of quaternion output weights.

λ : regularization parameter.

I : identity matrix of size $4L \times 4L$.

The resulting $\tilde{\beta}$ is then reassembled into quaternion form.

Relevance to machine learning

- **Compact Representations:** Hypercomplex numbers like quaternions allow for better parameterization, which decreases the number of parameters required to represent complex relationships. This is particularly helpful in deep learning models where reducing the number of parameters could decrease overfitting and increase computational performance.
- **Multidimensional Data Handling:** The immediate encoding of multidimensional data, such as multichannel images, e.g., RGB, or temporal sequences, without resorting to a high-dimensional tensor representation, is a natural task for hypercomplex numbers. It is very useful for applications like 3D image processing, where the application involves the use of quaternions to represent rotations in 3D space.
- **Efficient Rotational Symmetry Representation:** Hypercomplex numbers can directly model rotations and transformations, which are useful in fields like computer vision and graphics.

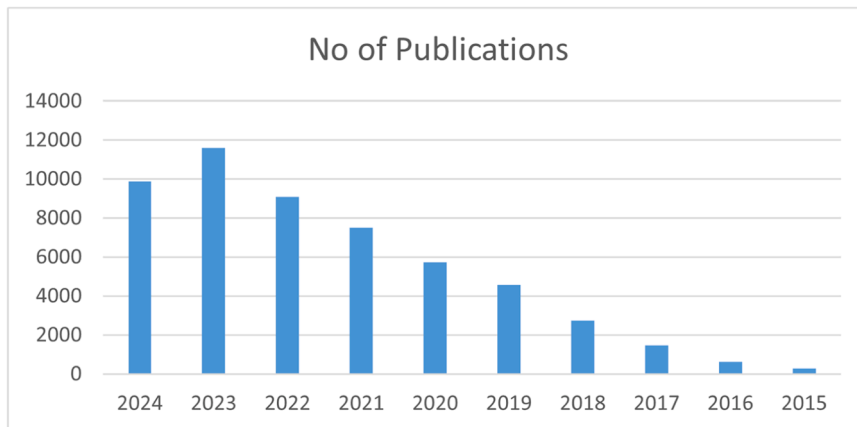


Fig. 10. IEEE explore publications.

Before we address subsequent review questions, we provide the statistics in number of publications published in three major databases viz., IEEE Explore, Springer and Elsevier. The search term includes “Quaternion” AND “Neural Network” OR “Deep learning” and includes research articles, book chapters, conference papers and magazines. Figs. 9–11 displays the search results.

The graph Fig. 10 shows a significant upward trend in the number of publications in IEEE Explore from 2015 to 2023, with a peak in 2023, followed by a slight decline in 2024.

It is evident from Fig. 11 which demonstrates consistency in the number of papers published each year. A peak occurred in 2024, with approximately 80 papers, demonstrating stability over time with some variations. This indicates the amount of research interest has gone up significantly over the decade, and it’s specifically increased recently.

As evidenced by Figs., 10, 11, and 12 the upward trend shows an increasing interest in hypercomplex numbers and their applications in machine learning, geometric algebra, and other computational methods. It coincides with the increased attention toward sophisticated mathematical frameworks such as Cayley-Dickson algebras and Clifford algebras, in solving complex high-dimensional problems that are encountered in areas such as deep learning, sequence modeling, and computer vision.

Significance

- **Growing Interest:** The number of publications has increased, especially in the last few years (2023–2024), indicating a growing interest among researchers in utilizing hypercomplex frameworks to address feature representation and computational efficiency challenges.
- **Relevance to Emerging Technologies:** The research is coincidental with the emergence of the leading-edge fields, Hypercomplex Neural Networks and Quaternion-based machine learning, which are vital to robotics, signal processing, and natural language processing.
- **Addressing Research Gaps:** The consistent growth indicates that past limitations in handling high-dimensional problems are being addressed by novel mathematical formulations and their practical implementations.
- **Interdisciplinary Impact:** The trend upwards reflects the interdisciplinary appeal of hypercomplex numbers, filling gaps between mathematics, physics, and AI for more robust and expressive models.

Question 2: What are the application areas, mathematical contributions, and limitations of past works that incorporates Quaternion numbers with Neural networks.

Table 2 displays past works on Quaternion neural networks

Synthesis of Table 2

The QNN research as a whole highlights application-motivated benefits (e.g., rotation/phase-informed representations and parameter efficiency) in addition to simple training pipelines with split activation and Hamilton-product layers. In the presence of head-to-head comparisons, quaternion models generally perform on par with or better than their mappable baselines with reduced parameter capacities, with robustness enhancements more pronounced in channel-couple-benefit applications (color, multi-channel audio, 3D pose). Yet, experimental verification is patchy across domains: numerous conclusions are restricted to small-to-mid-size datasets, and ablation procedures (initialization, activation type, quaternion product ordering) are still ad-hoc. Reproducing the research improves but is still compromised by inhomogeneous implementations. In general, the QNNs seem practically applicable in

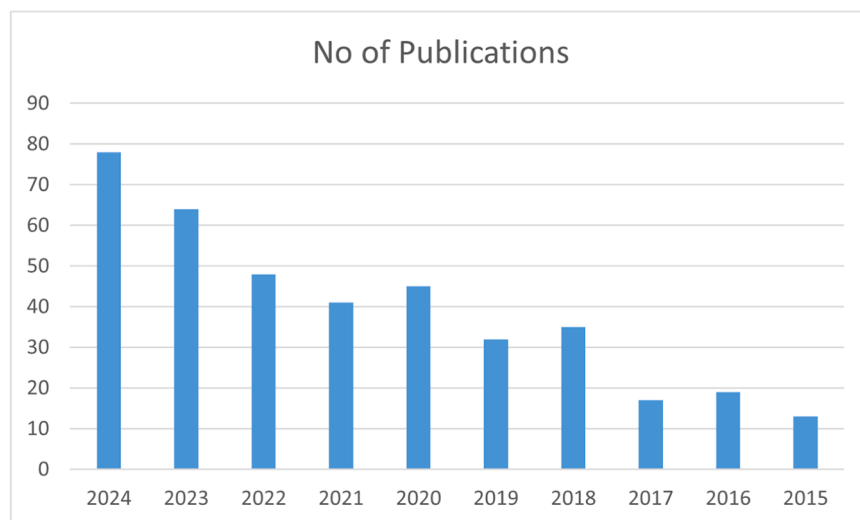


Fig. 11. Springer publications.

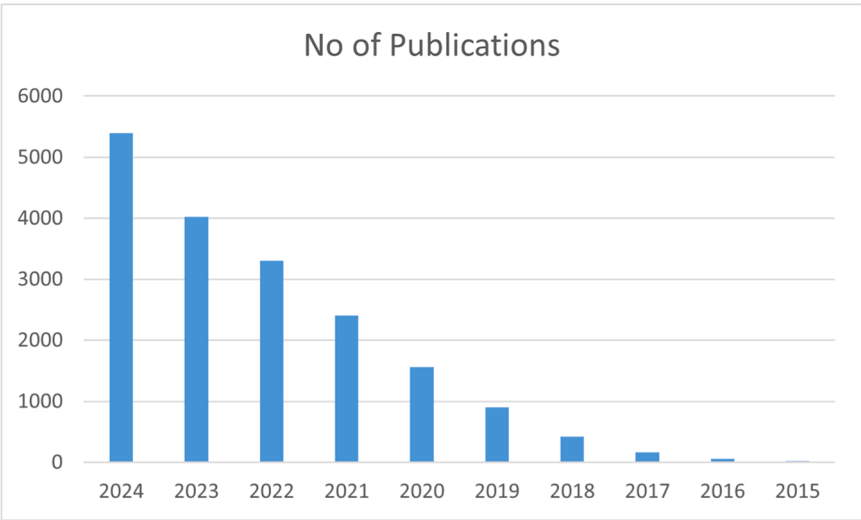


Fig. 12. Elsevier publications.

Table 2
Previous works on QNNs.

Author/Year	Application Area	Mathematical Contributions	Key Findings	Limitations
Hirose et al. [25]	Remote Sensing, Underground Radar, Mobile Communication	Quaternion Framework, Adaptive Processing, Fusion of Disciplines	Powerful Framework, Application Versatility, Physics Integration	Complexity of Implementation, Generalization Issues, Computational Demand
Huang, B., & Li, W. [26]	Color Image Compression, Geometric and Dynamical Properties, complex data analysis	Development of Neural Network Models, Gradient Flows on Manifolds, Geometric and Dynamical Analysis	Effectiveness of Neural Network Models, Geometric Insights, Numerical Experiments	Limited Scope, Complexity of Implementation, Numerical Stability
Chen et al. [27]	Medical Image Segmentation, Cross-Modality Data Integration, Potential for Broader Imaging Applications	Quaternion Representation, Quaternion Spatial-association Convolution, De-level Quaternion Cross-modality Fusion (De-QCF)	Effective Segmentation, Parameter Efficiency, Enhanced Spatial Learning	Complexity of Implementation, Generalizability, Computational Resources
Huang, Q., Ma, N., & Tu, Z. [28]	Neural Networks, Robotics and Automation, Time-Delay Systems	Lagrange Stability Analysis, Lyapunov Function Utilization, Quaternion Properties	Lagrange Stability Established, Innovative Analysis Method, Nonlinear Scalarization Approach	Failure to capture all dynamics of complex systems
Chen, H., Natsuaki, R., & Hirose, A. [29]	Wireless Communications, 3-D Data Processing, Artificial Intelligence in Communication	Quaternion Algebra, Symbol Detection Framework	Enhanced Detection Accuracy, 3-D Data Processing	Generalization to Other Modulations, Complexity of Implementation, Dependence on Channel State Information
Bill, J., Cox, B.A., & Champagne, L. E. [30]	High-Dimensional Problem Solving, Wireless Communications, Computer Vision	Quaternion Representation, Backpropagation Algorithms, Design of Experiments (DoE)	High Performance of QNN, Comparison of Algorithms, Statistical Improvement	Complexity of Implementation, Limited Application Scope
Fang et al. [31]	Recommender Systems, Collaborative Filtering, Complex Graph Modeling	Quaternion Space Utilization, Quaternion Embedding Propagation, Mean Pooling Strategy	Robustness to Noisy Graphs, Effective Feature Transformation, First Introduction of Quaternion Space	Vulnerability to Noisy Graphs, Complexity of Quaternion Space, Dependence on Initial Embeddings
Li, N., Cao, J., & Wang, F. [32]	Secure Communication Systems, Robotics and Control Systems, Networked Systems	Bipartite Secure Synchronization Criteria, Unitary Lyapunov Functional, Handling Structurally-Unbalanced Graphs	Bipartite Secure Synchronization, Structurally-Balanced and Unbalanced Graphs, Simulation Validation	Not suitable for more complex or dynamic network topologies, Random Deception Attacks
Shen et al. [33]	3D Point Cloud Processing, Robustness in Adversarial Settings, Classification Tasks	Rotation-equivariance proof, Gradient Equivariance, Training Invariance	Performance Improvement, Robustness to Adversarial Attacks, Gradient and Training Properties	Generalization to Other Domains, Computational Resources,
Xiao et al. [34]	Image Fusion Denoising, Dynamic Systems, Control Systems	Quaternion Real Representation, Fixed-Time Convergence, Integral Design Formula	Fixed-Time Convergence, Effective Noise Reduction, Chattering Phenomenon Addressed	Potential limitations in robustness, not generalize well to other types of problems or datasets, limiting its applicability

the presence of naturally 4-channel data or rotational symmetry, but comprehensive tests with standardized training recipes are required to support claims of generalization and robustness.

The summary of past works from Table 2 gives a good overview of the versatility and increasing use of quaternion-based frameworks across remote sensing, wireless communications, robotics, medical imaging, and neural networks. Some of the key observations made here are that quaternion algebra provides powerful tools for representing high-dimensional and complex data, thus improving computational efficiency, robustness, and performance in fields like 3D processing, secure communications, and collaborative filtering. Fig 13. Shows distribution of application areas in QNNs.

Fig. 13. illustrates the distribution of research studies applying hypercomplex frameworks across various domains. Image Processing and Remote Sensing & Communication emerge as the most popular areas, highlighting their adaptability to quantum-enhanced and hypercomplex methods. Emerging fields like 3D Point Cloud Processing and Dynamic Systems are also gaining attention. Fig. 14 displays mathematical contributions employed.

Fig. 14 showcases the major mathematical innovations employed in recent studies. Quaternion Representation and Quaternion Frameworks are the most used, indicating their central role in enhancing computational models. Other contributions, like Lyapunov Stability and Secure Synchronization, reflect efforts to improve stability and security in dynamic environments. Algorithm 3 provides QNN training loop.

Algorithm 3 outlines the training process for a quaternion-valued neural network, where inputs, weights, and biases are represented as quaternions. The forward pass employs the Hamilton product to capture inter-channel correlations across four dimensions, followed by a split activation function to preserve quaternion structure. The loss computation typically uses cross-entropy for classification tasks. The backward pass calculates quaternion-specific gradients via the Hamilton gradient rule, updating parameters with quaternion-aware optimizers. This approach efficiently encodes spatial transformations, making QNNs effective for 3D vision, speech processing, and robotics. Below we provide a complete step-by-step construction of a Quaternion Neural Network.

Step-by-step construction of a quaternion neural network

Quaternion neural networks extend standard neural nets by operating on quaternion-valued weights, inputs, and activations. A quaternion $q = a + b.i + c.j + d.k$ has four real components, enabling a compact encoding of multidimensional data (e.g. treating an RGB-color tuple as one quaternion) [20]. Below, we outline the construction of a QNN model step by step, highlighting how quaternion algebra is applied at each stage:

1. **Quaternion Representation of Data and Weights:** Each network variable (inputs, outputs, weights, biases) is represented as a quaternion. For example, a neuron's activation is $x = x_r + x_i.i + x_j.j + x_k.k$, and a weight is $W = W_r + W_i.i + W_j.j + W_k.k$, with $W_r, W_i, W_j, W_k \in \mathbb{R}$. Grouping real features into quaternions naturally embeds cross-channel relationships; for instance, four related real features can be treated as one quaternion feature, enforcing a form of weight sharing across those channels [52]. Initializing quaternion weights typically involves setting each of the four components (scalar and three imaginary parts) with random values (e.g. Gaussian or Xavier initialization), often ensuring they have equal variance so that no component dominates learning.
2. **Hamilton Product for Layer Computations:** QNNs replace the standard dot-product in layers with the Hamilton product (quaternion multiplication). In a fully connected layer, the pre-activation of an output quaternion neuron is computed as a weighted sum of input quaternions using Hamilton products. Likewise, a convolutional layer uses quaternion-valued kernels convolving over quaternion feature maps by Hamilton products in each local receptive field. The Hamilton product between two quaternions encodes component-wise interactions: given $q_1 = a_1 + b_1i + c_1j + d_1k$ and $q_2 = a_2 + b_2i + c_2j + d_2k$, their product $q_1 * q_2$ is itself a quaternion computed as:

$$q_1 * q_2 = (a_1a_2 - b_1b_2 - c_1c_2 - d_1d_2) + (a_1b_2 + b_1a_2 + c_1d_2 - d_1c_2)i + (a_1c_2 - b_1d_2 + c_1a_2 + d_1b_2)j + (a_1d_2 + b_1c_2 - c_1b_2 + d_1a_2)k$$

Each output component is a bilinear combination of input components and weight components. This operation inherently mixes the real and imaginary parts, capturing inter-channel couplings that real-valued layers would otherwise need separate parameters to learn. For instance, a quaternion convolutional layer on a color image can learn cross-channel features (corre-

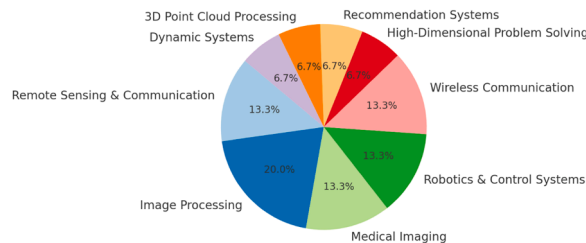


Fig 13. Application areas in QNNs.

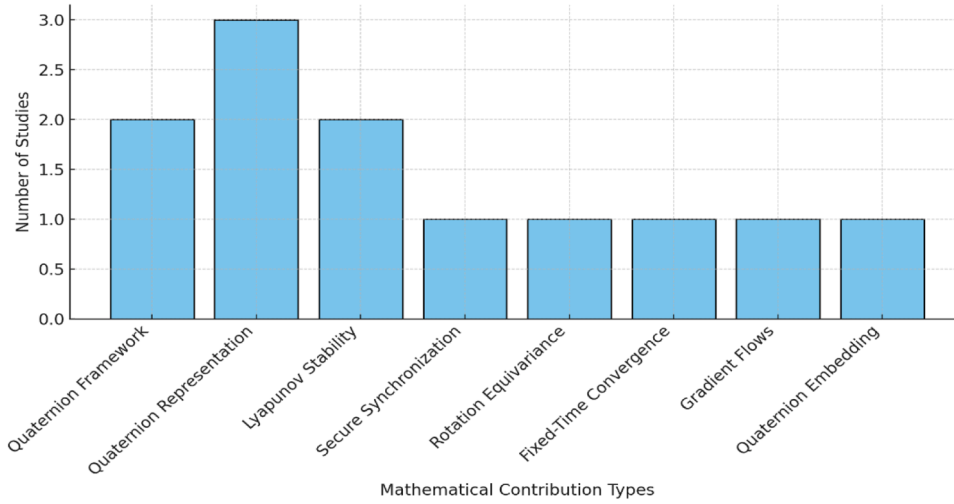


Fig 14. Focus on mathematical contributions.

Algorithm 3

QNN training loop.

Input: Training set $D = \{(Q_{in}, y)\}$, learning rate η , epochs E
Output: Trained quaternion weights W_q , biases b_q

```

1: Initialize  $W_q, b_q$  with quaternion-aware Xavier initialization
2: for epoch = 1 to  $E$  do
3:   for each  $(Q_{in}, y)$  in  $D$  do
4:     // Forward pass
5:      $Q_{mul} \leftarrow \text{HAMILTON\_PRODUCT}(Q_{in}, W_q)$  // quaternion multiplication
6:      $Q_{act} \leftarrow \text{ACT\_FN}(Q_{mul} + b_q)$  // activation (e.g., split ReLU)
7:      $y_{hat} \leftarrow \text{SOFTMAX}(Q_{act})$ 
8:
9:     // Loss computation
10:     $L \leftarrow \text{CROSS\_ENTROPY}(y_{hat}, y)$ 
11:
12:    // Backward pass
13:     $\delta \leftarrow \text{GRAD\_LOSS}(y_{hat}, y)$ 
14:     $\delta_{act} \leftarrow \text{GRAD\_ACT\_FN}(\delta, Q_{act})$ 
15:     $\partial W_q \leftarrow \text{HAMILTON\_GRAD}(Q_{in}, \delta_{act})$ 
16:     $\partial b_q \leftarrow \text{SUM}(\delta_{act})$ 
17:
18:    // Parameter update
19:     $W_q \leftarrow W_q - \eta * \partial W_q$ 
20:     $b_q \leftarrow b_q - \eta * \partial b_q$ 
21:  end for
22: end for

```

lations between R, G, B, etc.) through the Hamilton product of a quaternion filter with a quaternion patch of the image [53]. In practice, implementing a Hamilton product involves a fixed sequence of real multiplies and adds (Algorithm 4). Importantly, the Hamilton product is non-commutative, so weight order matters (i.e. $W * x \neq x * W$), but most QNN formulations use a consistent ordering (usually weight-on-left, input-on-right for layers).

3. **Split Activation Functions:** After the linear combination via Hamilton products, a nonlinearity is applied. The most common approach is a split activation, meaning the activation function is applied component-wise to each of the quaternion's four parts [54]. For example, if $o = u + v, i + w, j + z, k$ is a neuron's pre-activation quaternion output, a split ReLU would produce $\sigma(o) = \max(0, u) + \max(0, v)i + \max(0, w)j + \max(0, z)k$ (ReLU applied independently to u, v, w, z). This preserves the quaternion structure while introducing nonlinearity. Split activations (ReLU, sigmoid, tanh, etc.) are simple and ensure each component's gradient flows separately. More advanced quaternion-specific activations have been proposed (e.g. functions that modulate the quaternion's magnitude or phase as a whole), but the component-wise approach remains the standard due to its simplicity and proven effectiveness.
4. **Forward and Backward Propagation Workflow:** The forward pass of a QNN proceeds similarly to a real network, except using quaternion arithmetic. Each layer computes output quaternions from input quaternions using Hamilton products and adds a

Algorithm 4

Forward pass computation in a quaternion-valued fully connected layer using the Hamilton product and split activation.

```

# Forward pass for a quaternion dense layer
# Inputs: X[i] = (x[i]_r, x[i]_i, x[i]_j, x[i]_k) for i = 1...N (N quaternion inputs)
# Weights: W[j,i] = (W_r, W_i, W_j, W_k) for j = 1...M, i = 1...N (quaternion weight matrix)
# Bias: B[j] = (b_r, b_i, b_j, b_k) for j = 1...M (quaternion bias)
for each output neuron j = 1...M:
    # Compute Hamilton product sum: q_sum = sum_i W[j,i] * X[i]
    temp = (0, 0, 0, 0) # initialize quaternion sum
    for each input i = 1...N:
        # Hamilton product: W[j,i] * X[i]:
        a1, b1, c1, d1 = W[j,i]_r, W[j,i]_i, W[j,i]_j, W[j,i]_k # weight components
        a2, b2, c2, d2 = X[i]_r, X[i]_i, X[i]_j, X[i]_k # input components
        prod = (a1*a2 - b1*b2 - c1*c2 - d1*d2,
                a1*b2 + b1*a2 + c1*d2 - d1*c2,
                a1*c2 - b1*d2 + c1*a2 + d1*b2,
                a1*d2 + b1*c2 - c1*b2 + d1*a2) # quaternion product (u, v, w, z)
        temp = temp + prod # accumulate (element-wise addition of quaternion components)
    # Add bias and apply activation (split):
    o_j = temp + B[j]
    o_j = (σ(o_j^r), σ(o_j^i), σ(o_j^j), σ(o_j^k)) # apply activation to each part

```

quaternion bias. Pseudocode for a simple quaternion fully connected layer's forward pass is given below. We use indices i for input neurons and j for output neurons, and $[*]$ to denote quaternion components:

In [algorithm 4](#), each output quaternion o_j is computed by summing Hamilton products $W_{j,i} * x_i$ over all inputs i , then adding a quaternion bias B_j and applying an activation σ to each component. The backward pass (gradient computation) in QNNs follows from the same chain rule as real nets. The partial derivatives of a loss $\mathcal{L}$ with respect to each weight component (e.g. $\partial \mathcal{L} / \partial W_r$, $\partial \mathcal{L} / \partial W_i$, etc.) are obtained by backpropagating through the Hamilton product. Notably, the Hamilton product is a linear combination of products of components, so its gradients can be derived by treating it as a set of real operations (as seen in the pseudocode). Modern autodiff frameworks can automatically compute gradients for quaternion layers by breaking the operations into real multiplies and adds under the hood. In essence, during backpropagation a quaternion weight's four real components get updated via four corresponding gradients. The update rule (e.g. SGD) looks identical to the real case, just applied to each component of each quaternion weight. Thus, implementing quaternion backprop can leverage existing optimizers by handling quaternions as grouped real parameters.

5. Integration with Deep Learning Frameworks: QNN models can be integrated into PyTorch through custom layers or third-party libraries. This can be done by manually quantizing quaternion operations: i.e., express a quaternion as a length-4 vector and declare the Hamilton product formula as a function, and then use vanilla PyTorch autograd on those operations. Nevertheless, for easier development, there are specialized libraries offering pre-implemented quaternion layers. For example, hTorch extends PyTorch with a QuaternionTensor class and quaternion layers (QLinear, QConv, etc.), allowing one to write code like `model = nn.Sequential(QLinear(10,20), nn.ReLU(), QLinear(20,10))`. Under the hood, these layers handle the grouping of 4 real values per quaternion and perform Hamilton-product-based forward/backward computations.

Computational and memory complexity comparison

One key motivation for hypercomplex networks is the computational and parameter efficiency they can offer. By representing multiple real dimensions as a single hypercomplex number, QNNs, octonion NNs, and other Cayley-Dickson networks can drastically

Table 3

Parameter count and complexity by number system.

Network type	Parameter count (relative to real)	Ops per multiply (real multiplications + additions)	Memory usage (relative)
Real-valued network	$1 \times$ (baseline P parameters)	1 mult + 1 add per weight multiply (e.g. dot product)	$1 \times$ (baseline)
Complex-valued network (2D)	$\approx 0.5 \times$ (half the parameters) – one complex weight replaces 2 real weights	4 mult + 2 add per complex multiply (naive)	$\approx 0.5 \times$ (half)
Quaternion network (4D)	$\approx 0.25 \times$ (quarter the parameters) [55] – one quaternion replaces 4 reals	16 mult + 12 add per quaternion multiply (Hamilton product)	$\approx 0.25 \times$ (quarter)
Octonion network (8D)	$\approx 0.125 \times$ (one-eighth the parameters) – one octonion replaces 8 reals	64 mult + 56 add per octonion multiply (naive estimate)	$\approx 0.125 \times$ (one-eighth)
Cayley-Dickson 2n-D (general)	$1/(2^n)$ the parameters (in theory)	$\sim (2^n)^2$ mult + many adds (grows quadratically with dimension)	$1/(2^n)$ (for parameters)

reduce the number of independent parameters needed, albeit at some increase in per-operation cost. Table 3 summarizes the parameter count, floating-point operation cost, and memory footprint for networks in various domains (real, complex, quaternion, octonion), assuming we design each network to produce an equivalent real-output size for a fair comparison. (For example, a quaternion network with M quaternion outputs corresponds to $4M$ real outputs, comparable to a real network with $4M$ neurons in that layer.)

Parameter count: A quaternion network typically uses $4 \times$ fewer parameters than an equivalent real network. In general, using an n -dimensional hypercomplex algebra (of size $n = 2, 4, 8, \dots$) yields about a n -fold reduction in weight count for layers of comparable output size. This stems from the fact that one hypercomplex weight (e.g. one quaternion) encodes n real coefficients but is treated as a single unit that connects to n inputs simultaneously. For example, replacing a real fully-connected layer (with, say, D inputs and H outputs) by a quaternion layer allows grouping the D inputs into $D/4$ quaternion inputs and $H/4$ quaternion outputs; the weight matrix then has $(H/4) \times (D/4)$ quaternion entries (each with 4 components), which corresponds to only $1/4$ of the number of real parameters of the original layer.

Computational cost: The cost of using fewer weights is that each hypercomplex multiply is more costly than a real multiply. It requires 4 real multiplications and 2 adds to perform a single complex multiply (for two complex numbers), and quaternion multiply 16 real multiplications and 12 additions. Octonion multiply, similarly, will require 64 multiplications (and of the same order of additions) if performed directly. This $O(n^2)$ quadratic growth in operation with size n is a fundamental consequence of multiplication rules in the algebra. So, a quaternion layer with fewer than half the number of weights can still have roughly the same number of FLOPs as a real layer (because each weight's computation is $16 \times$ more costly than a real multiply).

Memory usage: The usage of memory (for model parameter storage) grows proportionally with real parameter size. Therefore, quaternion, complex, and octonion networks use about $1/2$, $1/4$, and $1/8$ of memory, respectively, of their corresponding real-valued networks, assuming all weights are being stored as regular floats. For example, converting a real model into quaternion form results in a 75 % reduction in model size (memory), and this is a real benefit for model deployment on constraint devices. Wu et al. [22] inform that their deep octonion networks (8D) preserved accuracy with significant compression of the model.

Question 3: What are the application areas, mathematical contributions, and limitations of past works that incorporate Octonion numbers with Neural networks.

Table 4 displays previous studies on Octonion neural networks.

Synthesis of Table 4

A clear pattern in ONN research is the prominence of stability and convergence analyses. This focus stems from octonions' non-associativity, which makes layer definitions order-sensitive and places stronger requirements on initialization, operator ordering,

Table 4
Previous works on ONNs.

Author/Year	Application area	Mathematical contributions	Key findings	Limitations
Cariow, A., & Cariowa, G. [35]	Signal Processing, Complex data representation	Fast Walsh-Hadamard transform.	Reduces the arithmetic complexity in octonionic neural networks	Does not address convergence issues or model interpretability, not suitable for large data sets
Li, B., Cao, Y., & Li, Y. [36]	Signal Processing, Image and Video Processing	Existence and Uniqueness, Global Exponential Stability, Non-Decomposition Method	Existence of Solutions, Uniqueness and Stability	Limit applicability to real-world scenarios, Focus on Specific Network Type
Kuroe, Y., Iima, H., & Maeda, Y. (2020) [37]	Complex Data Processing, Robotics and Control Systems, Quantum Computing	Extension of Hopfield Networks, Energy Function Construction	Four Models Proposed, Dynamic Behavior Analysis, Non-Associative Properties	Complexity of Octonions, Lack of Empirical Validation
Gao, J., & Dai, L. [38]	Signal Processing, Robotics	Existence of Solutions, Global Exponential Stability, Mixed Delay Analysis	Analyze the stability and existence of solutions in complex neural network models	Complexity of Octonions, Specific Conditions, Limited Scope
Wang, J., & Liu, X. (2021) [39]	Control Systems, Adaptive Systems, Signal processing	Finite Time Stability (FTS), Two-Phase Method (2PM), New Sign Function	Stability Analysis, Controller Design, Adaptive FTS	Assumption of Unbounded Delays, Complexity of Octonions
Hitzer, E. [40]	Applied in signal processing, particularly for reconstructing signals using new polar representations of octonion analytic signals	minimal embedding of octonions in geometric algebra, which helps express octonion multiplication's nonassociativity using a sum of up to four associative geometric algebra product terms	New polar representations of octonion analytic signals, along with novel signal reconstruction formulas that enhance the understanding of these signals	Challenges in practical implementations of the proposed signal reconstruction formulas
Bojesomo, A., Liatsis, P., & Almarzouqi, H. [41]	Marine Debris Detection, Deep Learning Applications, Remote Sensing	Octonion Neural Networks (ONNs), Higher-Dimensional Relationships	Improved Segmentation Accuracy, Parameter Efficiency, Effective Feature Representation	Resource Constraints, Dataset Specificity, Complexity of Octonions
Popa, C. (2017) [42]	Neural Network Design, Control systems	Exponential Stability Criteria, Extension of Existing Theories	Introduction of Delayed Octonion-Valued Networks, Numerical Validation	The reliance on linear matrix inequalities for stability criteria might limit the applicability to certain types of networks or conditions.

and gradient flow. As a result, many papers prioritize well-posed Ness (existence/uniqueness, boundedness, finite-time stability) before large-scale empirical evaluation. Where empirical results are provided, ONNs can exhibit strong representational capacity (richer cross-channel interactions), but training is more brittle without careful architectural constraints or real-valued embeddings that fix multiplication orders. The literature thus reveals a practical trade-off: ONNs offer high expressivity but demand stricter engineering (ordered products, stable normalizations, learning-rate control). Future work should pair the existing theoretical maturity with standardized benchmarks, report robustness under perturbations/rotations, and release reference implementations to close the gap between theory and practice.

The summarized works from Table 4 on octonions reveal their emerging potential in signal processing, robotics, control systems, and deep learning applications. The key mathematical contributions include frameworks for stability analysis, energy function construction, finite-time stability, and signal reconstruction formulas. Octonion neural networks have demonstrated promising results in improved segmentation accuracy, parameter efficiency, and higher-dimensional feature representation. However, challenges of adoption to the larger scale include non-associative properties which make the octonions complex, resource constraints, limited empirical validation, and applicability to real-world datasets. Fig. 14 shows key findings reported in ONNs.

Fig. 15 summarizes the major achievements reported across octonion neural network studies. Stability analysis emerges as the most common key finding, followed closely by improvements in segmentation accuracy and parameter efficiency, underscoring the focus on robust and efficient learning systems. Fig. 16 depicts the major application areas in ONN.

Also as evidenced from Fig 17 we can see the growing trend in publications w.r.t ONN.

Algorithm 5 provides ONN training loop.

As per algorithm 5 ONN training process of quaternion operations is extended to the 8-dimensional octonion space to enable more advanced cross-channel interactions. Strictly ordered evaluation octonion multiplication is used for the forward pass to limit non-associativity effects. Non-linear activation functions and regular softmax output layers are employed next. Octonion-sensitive gradients during the backward pass are calculated to treat non-associatively. Though ONNs have greater representational capacity than QNNs, they require careful initialization and training stability because of their higher dimensionality and larger computational complexity. They are applicable in image segmentation, 3D vision, and quantum-inspired learning settings.

Question 4. What are the application areas, mathematical contributions, and limitations of past works on Hypercomplex numbers.

As evidenced from Table 5 some of the research on hypercomplex numbers shows great applicability to the representation of quantum states, image processing, natural language processing, and mathematical modeling. Some key contributions in mathematics are the generalization of hypercomplex numbers, parameterized hypercomplex multiplication, hypercomplex operator calculus, and effective convolution rules. These yield enhanced class discrimination performance, higher parameter efficiency, more advanced processing of data, and greater flexibility of architecture across these different disciplines. However, limitations persist due to the complexity of implementation, computation overhead, scalability issues. Comparative studies consistently indicate that HNNs outperform real-valued networks in capturing multi-dimensional dependencies, and in some cases exceed complex-valued networks in representation efficiency. However, these gains often come at the cost of increased parameter tuning and computational demands, which motivates ongoing exploration of optimized training pipelines and hybrid designs.

Applications for quantum-aware models

Quaternion and other hypercomplex neural networks offer promising synergies with quantum computing paradigms. Below we discuss two concrete application areas grounded in recent research:

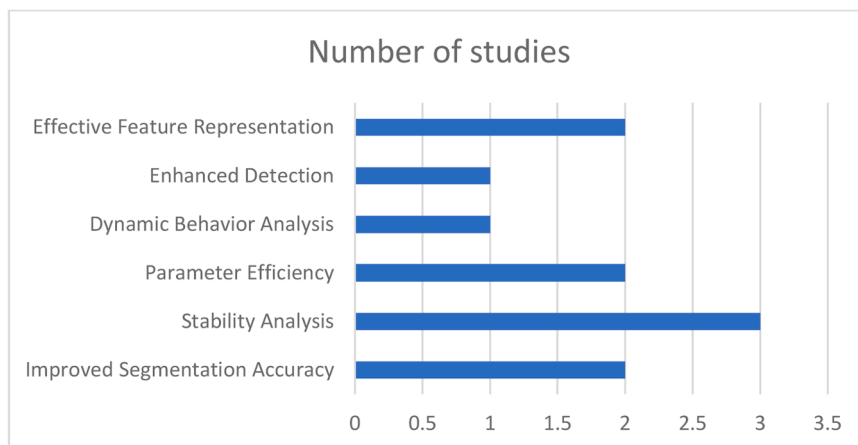


Fig. 15. Key findings.

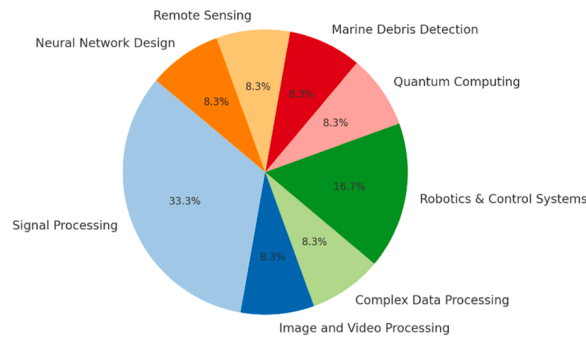


Fig 16. Application areas in ONN.

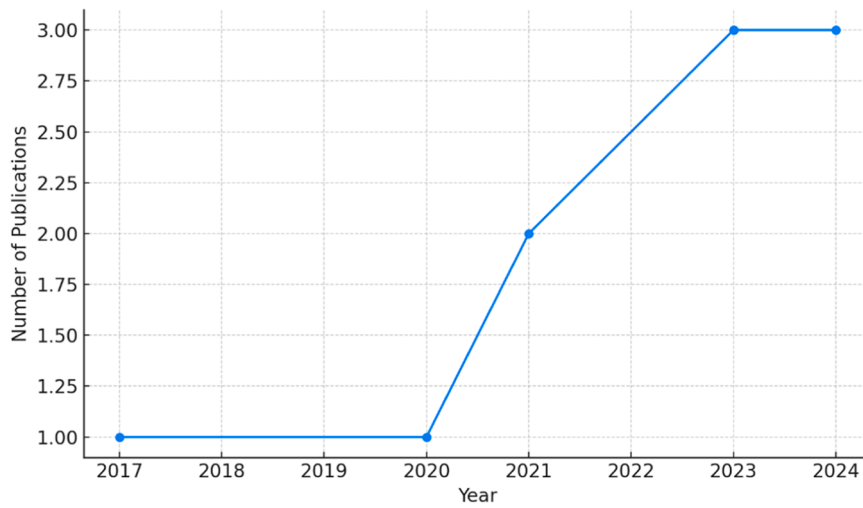


Fig 17. Publication trend in ONN research.

Algorithm 5

ONN training loop.

Input: Training set $D = \{(O_{in}, y)\}$, learning rate η , epochs E
Output: Trained octonion weights W_o , biases b_o

```

1: Initialize  $W_o, b_o$  with octonion-aware initialization
2: for epoch = 1 to  $E$  do
3:   for each  $(O_{in}, y)$  in  $D$  do
4:     // Forward pass with non-associativity handling
5:      $O_{mul} \leftarrow \text{OCTONION\_PRODUCT}(O_{in}, W_o)$  // define consistent multiplication order
6:      $O_{act} \leftarrow \text{ACT\_FN}(O_{mul} + b_o)$ 
7:      $y_{hat} \leftarrow \text{SOFTMAX}(O_{act})$ 
8:
9:     // Loss computation
10:     $L \leftarrow \text{CROSS\_ENTROPY}(y_{hat}, y)$ 
11:
12:    // Backward pass
13:     $\delta \leftarrow \text{GRAD\_LOSS}(y_{hat}, y)$ 
14:     $\delta_{act} \leftarrow \text{GRAD\_ACT\_FN}(\delta, O_{act})$ 
15:     $\partial W_o \leftarrow \text{OCTONION\_GRAD}(O_{in}, \delta_{act})$ 
16:     $\partial b_o \leftarrow \text{SUM}(\delta_{act})$ 
17:
18:    // Parameter update
19:     $W_o \leftarrow W_o - \eta * \partial W_o$ 
20:     $b_o \leftarrow b_o - \eta * \partial b_o$ 
21:  end for
22: end for

```

Table 5

Previous works on HNS.

Author/Year	Application area	Mathematical contributions	Key findings	Limitations
Singh, P., Gupta, A., & Joshi, S.D. (2024)	Quantum State Representation, Point Cloud Image Processing	Generalized Hypercomplex Numbers, Unified Multiplication Operation	Existence of Generalized Hypercomplex Numbers, Spherical Multiplication	Non-Distributive Nature, Complexity in Higher Dimensions,
Bojesomo, A., Liatsis, P., & Marzouqi, H.A. [43]	Spatiotemporal Data Processing, Multimodal Learning	Introduction of Hypercomplex Numbers, Design of Advanced Architectures	Superior Performance, Integration of Advanced Techniques	Complexity of Implementation, Computational Overhead
Vieira, N., Ferreira, M., Rodrigues, M.M., & Kraußhar, R.S. [44]	Boundary Value Problems, Inhomogeneous Problems	Hypercomplex Operator Calculus, Factorization of Operators	Explicit Fundamental Solutions, Conditions for Solutions	The explicit conditions for solutions may be sensitive to parameter variations.
Zhang, A., Tay, Y., Zhang, S., Chan, A., Luu, A.T., Hui, S.C., & Fu, J. [45]	Natural Language Processing, Text Style Transfer, Subject-Verb Agreement	Parameterized Hypercomplex Multiplication (PHM) Layer, Kronecker Product Utilization	Parameterization of Hypercomplex Multiplications, Architectural Flexibility	Complexity in Learning, Dependence on Hyperparameter
Grassucci, E., Zhang, A., & Communiello, D. [46]	Image and audio processing	Hypercomplex Parameterization, Efficient Convolution Rules	Parameter Reduction, Flexibility in Domains	Scalability Concerns, Domain Dependence
Shahadat, N., & Maida, A. [47]	Image Classification, Computer Vision, Multimedia Processing	Integration into ResNet Architectures, Parameterized Hypercomplex Multiplication (PHM)	Improved Classification Accuracy, Applicability Across Datasets	Focus on Backend Only, Complexity of Implementation
Grassucci, E. et al. [48]	Audio Datasets, Multidimensional Inputs	Parameterized Hypercomplex Convolutions, Reduction of Parameters	Performance Improvement, Flexibility Across Domains	Complexity in Implementation, Computational Overhead
Cariow, A., & Finko, O. [49]	Deep learning, Data Mining, Error Correction	Resource-Saving Algorithms, Polynomial Residue Number System	Advancement in Data Processing, Error Correction Innovations	Complexity of Implementation, Energy and Hardware Constraints
Kalinovskiy, Y., & Boiarinova, Y. [50].	Mathematical Modeling, Symbolic Computation	Exponential Function Representation, Hypercomplex Computing Software	perform complex symbolic operations, fragments of programs and outcomes from symbolic calculations	The reliance on hypercomplex computing software (HCS) could limit accessibility for users unfamiliar with this software
Kalinovsky, Y.O., Lande, D.V., Boyarinova, Y.E., & Turenko, A.S. [51].	advanced mathematical modeling , quantum mechanics	hypercomplex number systems (HNS) of 4-dimension	enhancing computational techniques, identification of zero dividers, contributing to the theoretical framework	Does not address potential applications or implications of the non-commutative hypercomplex number systems in real-world scenarios

1. Quaternionic Qubits and Quantum Neural Architectures

Altamirano-Escobedo and Bayro-Corrochano (2023) [56] introduce the Quaternionic Quantum Neural Network (QQNN), where both inputs and targets are encoded as quaternionic qubits. This design introduces activation operators within the quaternion space and attains higher classification performance than conventional real-valued networks on benchmark data, particularly on pattern recognition tasks. With their quaternion unit representation of qubits, QQNNs give compact descriptions of quantum states and state transitions, providing a structured and expressive paradigm consistent with quantum computation principles.

2. Quaternion Models for Quantum Learning Frameworks

Talebi et al. (2025) [57] propose a learning framework for quantum information processing machines, leveraging quaternion division algebra and HR-calculus. They develop adaptation and optimization strategies for quantum learning units, that are their analogs in the quantum domain, and achieve mathematically tractable solutions that possess provably convergent properties. This indicates the possibility of embedding quaternion-enriched learning dynamics in quantum-sensitive architectures.

Role and value to domain researchers

- **Structured Quantum Representation:** Quaternions extend naturally complex-valued representation (which is inherently compatible with quantum mechanics) to enable more robust geometric encoding of quantum states and operations.
- **Compact Parameterization:** Embedding qubits in four-dimensional algebra, QQNNs eliminate redundancy and optimize representational efficiency.
- **Theoretical Rigour:** Quaternion learning models such as that of Talebi et al. can be subjected to formal proofs of convergence, offering a firm foundation for the construction of quantum-aware learning systems.

Observations and future directions

Below are some of the observations made from our analysis of the literature.

Quaternion neural networks

- QNNs demonstrates applicability versatility in remote sensing, medical imaging, robotics, and communication systems.
- Provide parameter efficiency, better segmentation, and the robustness of adversarial attack.
- Mathematical innovations include quaternion convolutions, Lyapunov stability analysis, and rotation-equivariant proofs.
- The techniques also suffer from high computational demands, complexity of implementation, and challenges in generalization to new domains.

Octonion neural networks

- Focus on niche areas like signal processing, robotics, and quantum computing.
- Allow for efficient high-dimensional data representation, stability analysis, and signal reconstruction.
- Struggle with non-associativity, thus making mathematical formulations complicated.
- These are issues of limited real-world validation, resource constraints, and difficulty scaling for large datasets.

It was also observed that the number of publications on Octonion numbers is limited, and it may be due to the following reasons:

- Non-Associativity: Octonions are non-associative, meaning that their mathematical formulations are difficult.
- Complexity: High computational and mathematical complexity makes the implementation challenging.
- Niche Applications: Most octonion research remains theoretical, with fewer practical real-world applications.
- Resource Constraints: ONNs are computationally intensive, thereby limiting their scalability.
- Lack of Validation: Empirical validations and standardized frameworks for octonion networks are very few.

Hypercomplex number systems

- Used in a wide field of applications, including quantum mechanics, NLP, computer vision, and spatiotemporal data processing.
- Provide innovations like generalized hypercomplex multiplication, operator calculus, and parameter-efficient convolutions.
- Show strength in parameter reduction, advances in computing, and versatility in domain applications.
- Scalability issues, high computational overhead, and sensitivity to hyperparameters.

Table 6. displays Categorization of limitations in hypercomplex neural networks in terms of computational complexity (parameter/memory overhead), numerical complexity (gradient stability, non-associativity), and practical complexity (interpretability, reproducibility)

Research gaps

- The non-associativity of octonions brings some mathematical and computational complications along with them, making their realization very challenging in any real-world model of machine learning.
- Empirical validations and practical applications of octonion-based neural networks are a little scarce, and the theoretical aspects have been dealt with in most studies.
- Octonions also present resource limitations that forbid the scalability of large datasets for complex real-world applications.
- Most of the quaternion-based models, like other hypercomplex systems, usually do not provide generalization to broader domains of applications, like dynamic systems or diverse datasets.
- Although quaternion systems are applied effectively to multimodal data fusion, the idea of spatiotemporal learning with higher-order relationship of data has not yet been exploited for octonions and sedenions.
- While in a controlled environment, hypercomplex models have indeed shown great promise, there is a problem with scaling them up to bigger systems or datasets due to the bounded computational resources.

Software ecosystem and implementation challenges

While theory has evolved fast, hypercomplex neural networks (HNNs) are poorly supported by mainstream deep learning tools.

Table 6
Categorization of limitations in hypercomplex neural networks.

Limitation type	Description
Computational Overhead	Embeddings require $4 \times$ or $8 \times$ expansion compared to real-valued networks [44]
Numerical Instability	Octonion non-associativity makes computational pipelines order-sensitive [9]
Convergence Difficulty	Complex/octonion networks require careful training scheduling; slower to converge [22]
Interpretability	Few methods (e.g., PHB-cos transform) offer interpretability; most HNNs remain opaque [59]
Validation Gaps	Evaluations limited to mid-scale datasets; lack large-scale or robustness benchmarks

Several third-party libraries offer limited capability, but none now form a full standard ecosystem:

- **Quaternion Layers in PyTorch** — Pytorch libraries like `pyquaternion-nn` and Parcollet's Quaternion Neural Networks (QNNs) for PyTorch provide quaternion-valued dense, convolutional, and recurrent layers with Hamilton product versions and weight initializations. These libraries are used widely on vision and speech tasks but only quaternion algebra and usually a release behind the current PyTorch.
- **Octonion Implementations** — Octonion-valued layers have been prototyped (Wu et al., 2020) [22] but exist only as unstable, widely scattered libraries. Most of the implementations are employing ad-hoc real-valued embeddings (8-channel expansion) hardcoded into PyTorch/TensorFlow, with no specific autograd operators for non-associative multiplication.
- **Geometric Deep Learning Toolkits** — Toolkits like PyTorch Geometric and Geometric Algebra Layers from DeepMind offer Clifford/Geometric algebra operations, which are conceptually analogous to Cayley–Dickson constructions. They are not tailored towards quaternion/octonion algebras per se and must be adapted.

Although these first steps have been made, there are some impediments which prevent the emergence of hypercomplex neural networks on scale. To begin with, the lack of a common API to current libraries closes off result reproducibility as well as systematic benchmarks. Secondly, PyTorch and TensorFlow autograd engines do not naturally support octonion multiplication's non-associativity, so backpropagation is order-sensitive and must be handled by the user with caution. Third, they mainly work with real-valued embeddings that are associated with enormous overhead—double computational and memory overhead for complex numbers, quadruple for quaternions, and octuple for octonions. Lastly, most packages out there are research-grade with abysmal documentation, poor community support, and patchy long-term maintenance, which together constrain their utility and practical applications among researchers and practitioners.

Future directions

- **Scalability of Octonion Models.** Scalability of octonion neural networks is one of the most daunting challenges, which is managing non-associativity and the resultant computational cost. Future work should seek associativity-preserving surrogate operators or ordered computation schemes with stable gradient flow, as well as parallelized GPU/TPU kernels for fast octonion products. Such advances would render octonion networks computationally feasible for big-data applications like 3D data processing, robotics, and real-time scene understanding.
- **Unified Hypercomplex Frameworks.** Rather than distinct quaternion or octonion models, one of the most exciting approaches is to develop hybrid frameworks that can natively accommodate several hypercomplex systems (e.g., quaternions, octonions, sedenions) within a unified framework. Unified toolboxes might enable multimodal data fusion and spatiotemporal sequence learning with modular operators and hierarchical feature extraction from heterogeneous domains.
- **Domain-Agnostic Methodologies.** To increase flexibility, the next generation of research should design domain-agnostic training pipelines that are not based on explicit inductive biases like rigid channel counts. For instance, hypercomplex tensor decomposition and regularization techniques might provide strong performance for all tasks from dynamic control systems to natural language modeling with zero task-specific channel structure needs.
- **Empirical Verification and Industrial Translation.** As much of the existing literature is theoretical, it is imperative to conduct benchmark-based empirical studies that assess HNNs using adversarial robustness testing, secure communication protocols, and real-time decision-making scenarios. Industry collaboration will be essential in proving practicality in healthcare diagnostics, robotic sensing, and emerging wireless communications.
- **Lightweight Architectures and Open-Source Tools.** Future work should be in lightweight algorithmic designs (e.g., low-rank factorization of hypercomplex weights) combined with hardware-aware optimizations. At the same time, developing open-source hypercomplex libraries with standard APIs for PyTorch/TensorFlow would spur adoption and reproducibility, closing the theory-practice gap.
- **Extensions Beyond Octonions.** Generalization of Cayley–Dickson constructions to sedenions and Clifford algebraic structures with more general algebras is largely an open problem. It needs here, apart from overcoming mathematical problems of non-invertibility and loss of division properties, to explore their promise in quantum-conscious models, multidimensional signal fusion, and higher geometric learning. These directions would significantly enhance the expressiveness of hypercomplex neural architectures.

These directions address the prevailing issues while opening more of the potential of hypercomplex systems for further developments in machine learning and its applications. Researchers can take part in these directions to boost innovation and lay down frameworks for the future.

Conclusion

This work points to the emergent relevance of hypercomplex numbers, including quaternions and octonions, together with higher dimensional algebras, in attacking modern problems in machine learning. The relevant literature shows how these mathematical systems can very efficiently present multi-dimensional information, lead to parameter reductions, and improve dynamics of domains that include signal processing, image classification, robotics, and natural language processing. The research underscores several key insights,

- Quaternions are developed and applied properly in different application spaces, such as 3D data processing, geometric learning, communications, based on the balance of mathematical freedom and computational practicability.
- Octonions, though promising for higher-dimensional representations, face major problems in the non-associative nature, high resource demands, and fewer real-world applications.
- Hypercomplex systems offer a foundation for extending the capabilities of neural networks, especially in parameter efficiency, stability analysis, and multimodal learning.

Future work could focus on main gaps, such as scalability, ability to generalize to other types of domains, computational efficiency, development of standardized tools, and empirical validations. With these future challenges, hypercomplex systems can be a game-changer in the landscape of machine learning by offering innovative perspectives on complex problems in high-dimensional settings. This paper provides necessary groundwork for further exploration in the integration of hyper complexity in real-world applications but encourages interdisciplinary collaboration for its final realization. A key research gap in this study is the absence of comprehensive cross-model benchmarks directly comparing real, complex, quaternion, octonion, and Clifford networks across standardized datasets and tasks.

Ethics statements

No data was collected from any social media platforms.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

References

- [1] Lewis, A.C. (2005). William Rown Hamilton, lectures on quaternions (1853). Landmark writings in Western mathematics 1640–1940, 460–469. <https://doi.org/10.1016/b978-044450871-3/50116-9>.
- [2] T. Parcollet, Y. Zhang, M. Morchid, C. Trabelsi, G. Linares, R. De Mori, Y. Bengio, Quaternion convolutional neural networks for end-to-end automatic speech recognition, Interspeech 2018 (2018), <https://doi.org/10.21437/interspeech.2018-1898>.
- [3] D. Comminiello, M. Lella, S. Scardapane, A. Uncini, Quaternion convolutional neural networks for detection and localization of 3D sound events, in: ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2019, pp. 8533–8537, <https://doi.org/10.1109/icassp.2019.8682711>.
- [4] C.J. Gaudet, A.S. Maida, Deep quaternion networks, in: 2018 International Joint Conference on Neural Networks (IJCNN), 2018, pp. 1–8, <https://doi.org/10.1109/ijcnn.2018.8489651>.
- [5] Singh, P., Gupta, A., & Joshi, S.D. (2022). On the hypercomplex numbers of all finite dimensions: beyond quaternions and octonions. <https://doi.org/10.36227/techrxiv.19345820.v1>.
- [6] T. Parcollet, Y. Zhang, M. Morchid, C. Trabelsi, G. Linares, R. de Mori, Y. Bengio, Quaternion convolutional neural networks for end-to-end automatic speech recognition, Interspeech 2018 (2018), <https://doi.org/10.21437/interspeech.2018-1898>.
- [7] A. Muppidi, M. Radfar, Speech emotion recognition using quaternion convolutional neural networks, in: ICASSP 2021 - 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2021, pp. 6309–6313, <https://doi.org/10.1109/icassp39728.2021.9414248>.
- [8] J. Baez, The octonions, Bull. Am. Math. Soc. 39 (2) (2001) 145–205, <https://doi.org/10.1090/s0273-0979-01-00934-x>.
- [9] J.B. Kuipers, Quaternions and Rotation Sequences: a Primer with Applications to Orbits, Aerospace, and Virtual Reality, Princeton University Press, 1999.
- [10] T. Parcollet, M. Morchid, G. Linares, R.D. Mori, Bidirectional quaternion long short-term memory recurrent neural networks for speech recognition, in: ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2019, pp. 8519–8523, <https://doi.org/10.1109/icassp.2019.8683583>.
- [11] A. Hirose, F. Shang, Y. Otsuka, R. Natsuaki, Y. Matsumoto, N. Usami, Y. Song, H. Chen, Quaternion neural networks: a physics-incorporated intelligence framework [Hypercomplex signal and image processing], IEEE Signal. Process. Mag. 41 (3) (2024) 88–100, <https://doi.org/10.1109/msp.2024.3384179>.
- [12] L. Blaszczyk, Octonion spectrum of 3D octonion-valued signals - properties and possible applications, in: 2018 26th European Signal Processing Conference (EUSIPCO), 2018, pp. 509–513, <https://doi.org/10.23919/eusipco.2018.8553228>.
- [13] C. Popa, Octonion-valued neural networks. Lecture Notes in Computer Science, 2016, pp. 435–443, https://doi.org/10.1007/978-3-319-44778-0_51.
- [14] A. Micali, R. Boudet, J. Helmstetter, Clifford Algebras and their Applications in Mathematical Physics, Springer Science & Business Media, 2013.
- [15] D. Hestenes, G. Sobczyk, Clifford Algebra to Geometric Calculus: a unified Language for Mathematics and Physics, Springer Science & Business Media, 1984.
- [16] J.M. Selig, Clifford algebra and robot dynamics. Handbook of Geometric Computing, 2005, pp. 657–678, https://doi.org/10.1007/3-540-28247-5_20.
- [17] G. Dixon, On quaternions and octonions: their geometry, arithmetic, and symmetry, Math. Intell. 26 (2004) 75–77.
- [18] R.S. Kraußhar, Function theories in Cayley-Dickson algebras and number theory, Milan J. Math. 89 (1) (2021) 19–44, <https://doi.org/10.1007/s00032-021-00325-y>.
- [19] Altamirano-Gómez, G.E., & Gershenson, C. (2023). Quaternion convolutional neural networks: current advances and future directions. ArXiv, abs/2307.08663.
- [20] W. Shen, B. Zhang, S. Huang, Z. Wei, Q. Zhang, 3D-rotation-equivariant quaternion neural networks, in: A. Vedaldi, H. Bischof, T. Brox, J.M. Frahm (Eds.), Computer Vision – ECCV 2020. ECCV 2020. Lecture Notes in Computer Science, Computer Vision – ECCV 2020. ECCV 2020. Lecture Notes in Computer Science, 12365, Springer, Cham, 2020, https://doi.org/10.1007/978-3-030-58565-5_32.
- [21] J. Wu, L. Xu, F. Wu, Y. Kong, L. Senhadji, H. Shu, Deep octonion networks, Neurocomputing 397 (2020) 179–191, <https://doi.org/10.1016/j.neucom.2020.02.053>.
- [22] Ali, R., Kulyte, P., Borde, H.S., Lio, P. (2024). Metric Learning for Clifford Group Equivariant Neural Networks. ArXiv, arXiv preprint arXiv:abs/2407.09926. Proceedings of the Geometrygrounded Representation Learning and Generative Modeling (GRaM) at the 41 st International Conference on Machine Learning, Vienna, Austria. PMLR 251, 2024.

- [23] V.-H. Tran, T.N. Vo, T.T. Huu, T.M. Nguyen, A clifford algebraic approach to $e(n)$ -equivariant high-order graph neural networks, 2024 arXiv preprint arXiv: 2410.04692, 2024c.
- [24] G. Vieira, M. Eduardo Valle, Extreme learning machines on Cayley-Dickson algebra applied for color image auto-encoding, in: 2020 International Joint Conference on Neural Networks (IJCNN), 2020, pp. 1–8, <https://doi.org/10.1109/ijcnn48605.2020.9207495>.
- [25] A. Hirose, F. Shang, Y. Otsuka, R. Natsuaki, Y. Matsumoto, N. Usami, H. Chen, Quaternion neural networks: a physics-incorporated intelligence framework [Hypercomplex signal and image processing], IEEE Signal. Process. Mag. 41 (3) (2024) 88–100, <https://doi.org/10.1109/msp.2024.3384179>.
- [26] B. Huang, W. Li, Neural network models for the quaternion singular value decompositions, Appl. Math. Model. 129 (2024) 780–805, <https://doi.org/10.1016/j.apm.2024.02.019>.
- [27] J. Chen, G. Huang, X. Yuan, G. Zhong, Z. Zheng, C. Pun, J. Zhu, Z. Huang, Quaternion cross-modality spatial learning for multi-modal medical image segmentation, IEEE J. Biomed. Health Inform. 28 (2023) 1412–1423.
- [28] Q. Huang, N. Ma, Z. Tu, Lagrange stability of quaternion-valued memristive neural networks, Math. Methods Appl. Sci. (2024).
- [29] H. Chen, R. Natsuaki, A. Hirose, Robust symbol detection based on quaternion neural networks in wireless polarization-shift-keying communications, IEEE Trans. Neural Netw. Learn. Syst. 35 (2023) 16064–16075.
- [30] J. Bill, B.A. Cox, L.E. Champagne, A comparison of quaternion neural network backpropagation algorithms, Expert Syst. Appl. 232 (2023) 120448.
- [31] Y. Fang, P. Zhao, G. Liu, Y. Liu, V.S. Sheng, L. Zhao, X. Zhou, Quaternion-based graph convolution network for recommendation, World Wide Web 26 (2021) 2835–2854.
- [32] N. Li, J. Cao, F. Wang, Bipartite secure synchronization criteria for coupled quaternion-valued neural networks with signed graph, Neural Netw. Off. J. Int. Neural Netw. Soc. 180 (2024) 106717.
- [33] W. Shen, Z. Wei, Q. Ren, B. Zhang, S. Huang, J. Fan, Q. Zhang, Interpretable rotation-equivariant quaternion neural networks for 3D point cloud processing, IEEE Trans. Pattern Anal. Mach. Intell. 46 (2024) 3290–3304.
- [34] L. Xiao, P. Cao, Z. Wang, S. Liu, A novel fixed-time error-monitoring neural network for solving dynamic quaternion-valued Sylvester equations, Neural Netw. Off. J. Int. Neural Netw. Soc. 170 (2023) 494–505.
- [35] A. Cariow, G. Cariowa, Fast algorithms for deep octonion networks, IEEE Trans. Neural Netw. Learn. Syst. 34 (2021) 543–548.
- [36] B. Li, Y. Cao, Y. Li, The dynamics of octonion-valued neutral type high-order Hopfield neural networks with D operator, J. Intell. Fuzzy Syst. 44 (2023) 9599–9613.
- [37] Y. Kuroe, H. Iima, Y. Maeda, Four models of hopfield-type octonion neural networks and their existing conditions of energy functions, in: 2020 International Joint Conference on Neural Networks (IJCNN), 2020, pp. 1–7.
- [38] J. Gao, L. Dai, Weighted Pseudo almost Periodic Solutions of Octonion-Valued Neural Networks with Mixed Time-Varying Delays and Leakage Delays, AIMS Mathematics, 2023.
- [39] J. Wang, X. Liu, Finite time stability of octonion-valued neural networks with delays, in: 2021 36th Youth Academic Annual Conference of Chinese Association of Automation (YAC), 2021, pp. 55–60.
- [40] E. Hitzer, Embedding of octonion fourier transform in geometric algebra of $\mathbb{R}^3$ and polar representations of octonion analytic signals in detail, Math. Methods Appl. Sci. 47 (14) (2023) 11405–11421, <https://doi.org/10.1002/mma.9639>.
- [41] A. Bojesomo, P. Liatsis, H. Almarzouqi, Marine debris segmentation using a parameter efficient octonion-based architecture, IEEE Geosci. Remote Sens. Lett. 20 (2023) 1–5.
- [42] C. Popa, Exponential stability for delayed octonion-valued recurrent neural networks. Lecture Notes in Computer Science, 2017, pp. 375–385, https://doi.org/10.1007/978-3-319-59153-7_33.
- [43] A. Bojesomo, P. Liatsis, H.A. Marzouqi, Deep hypercomplex networks for spatiotemporal data processing: parameter efficiency and superior performance [Hypercomplex signal and image processing], IEEE Signal Process. Mag. 41 (2024) 101–112.
- [44] N. Vieira, M. Ferreira, M.M. Rodrigues, R.S. Kraußhar, Hypercomplex operator calculus for the fractional Helmholtz equation, Math. Methods Appl. Sci. (2024).
- [45] Zhang, A., Tay, Y., Zhang, S., Chan, A., Luu, A.T., Hui, S.C., & Fu, J. (2021). Beyond fully-connected layers with quaternions: parameterization of hypercomplex multiplications with $1/n$ parameters. ArXiv, abs/2102.08597.
- [46] Grassucci, E., Zhang, A., & Communiello, D. (2021). Lightweight convolutional neural networks by hypercomplex parameterization. ArXiv, abs/2110.04176.
- [47] N. Shahadat, A. Maida, Enhancing ResNet image classification performance by using parameterized hypercomplex multiplication, in: 2023 26th International Conference on Computer and Information Technology (ICCIT), 2023, pp. 1–6.
- [48] E. Grassucci, A. Zhang, D. Communiello, PHNNs: lightweight neural networks via parameterized hypercomplex convolutions, IEEE Trans. Neural Netw. Learn. Syst. 35 (2021) 8293–8305.
- [49] A. Cariow, O. Finko, Special issue: real, complex and hypercomplex number systems in data processing and representation, Appl. Sci. (2023).
- [50] Y. Kalinovsky, Y. Boiarinova, Method for representing an exponent in a fifth-dimensional hypercomplex number systems using a hypercomplex computing software, Elektron. Model. (2021).
- [51] Kalinovsky, Y.O., Lande, D.V., Boyarinova, Y.E., & Turenko, A.S. (2014). Computing characteristics of one class of non-commutative hypercomplex number systems of 4-dimension. arXiv: General Mathematics.
- [52] J.B. Kuipers, Quaternions and Rotation sequences: a Primer with Applications to Orbits, Aerospace, and Virtual Reality, Princeton University Press, 1999.
- [53] Altamirano-Gómez, G., & Gershenson, C. (2024). Statistical analysis of the impact of quaternion components in convolutional neural networks. *arXiv preprint arXiv:2409.00140*.
- [54] S.M. Iqbal, S. Mishra, Neural networks at a fraction with pruned quaternions, in: Proceedings of the 6th Joint International Conference on Data Science & Management of Data (10th ACM IKDD CODS and 28th COMAD), 2023, pp. 19–27.
- [55] G. Altamirano-Escobedo, E. Bayro-Corrochano, Quaternion quantum neural network for classification, Adv. Appl. Clifford Algebr. 33 (4) (2023) 40.
- [56] M. Yu, C. Bai, J. Yu, M. Zhao, T. Xu, H. Liu, R. Yu, Translation-based embeddings with octonion for knowledge graph completion, Appl. Sci. 12 (8) (2022) 3935.
- [57] C.A. Popa, Asymptotic and Mittag-Leffler synchronization of fractional-order octonion-valued neural networks with neutral-type and mixed delays, Fract. Fract. 7 (11) (2023) 830.
- [58] T. Parcollet, M. Ravanelli, M. Morchid, G. Linarès, C. Trabelsi, R. De Mori, Y. Bengio. Quaternion recurrent neural networks, 2018 arXiv preprint arXiv: 1806.04418.
- [59] E. Lopez, E. Grassucci, D. Capriotti, D. Communiello, Towards explaining hypercomplex neural networks, in: 2024 International Joint Conference on Neural Networks (IJCNN), IEEE, 2024, pp. 1–8.